

Lab # 1

Client-side technologies: HTML-I

OBJECTIVE:

To understand the basic structure of HTML, its elements, and their applications for creating a personal site.

THEORY:

HTML (HyperText Markup Language) is the standard markup language used to create web pages. It provides the skeleton structure of the web document and allows developers to define the content and its arrangement.

1. The Anatomy of an HTML Tag:

HTML tags are the building blocks of web pages. Each tag is a pair of angled brackets enclosing an element's name.

2. What We're Building - HTML Personal Site:

A personal website typically contains the following:

- A homepage.
- Sections like biography, skills, and projects.
- Images and hyperlinks for navigation.

Example Code for a Simple Homepage:

```
<> abc.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  |   <title>My Personal Website</title>
5  </head>
6  <body>
7  |   <h1>Welcome to My Personal Site!</h1>
8  |   <p>Hello! I am [Your Name]. This is a simple webpage built using HTML.</p>
9  </body>
10 </html>
```

3. What is The HTML Boilerplate?

The HTML boilerplate refers to the basic template of an HTML file.

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9
10 </body>
11 </html>
```

4. HTML Lists

Lists are used to group related items. There are two main types:

- Ordered Lists (): Items are numbered.
- Unordered Lists (): Items use bullets.

```
1  <!-- ... -->
2
3  <!-- ... -->
4
5  <!-- ... -->
6
7  <!-- ... -->
8  <body>
9      <h3>My Favorite Fruits:</h3>
10     <ul>
11         <li>Apple</li>
12         <li>Banana</li>
13         <li>Cherry</li>
14     </ul>
15
16     <h3>Steps to Cook:</h3>
17     <ol>
18         <li>Preheat the oven.</li>
19         <li>Mix the ingredients.</li>
20         <li>Bake for 30 minutes.</li>
21     </ol>
22
23 </body>
24 </html>
```

5. HTML Image Elements

The `<img>` tag is used to add images to your website.

```

```

- `src`: Specifies the image source (file path or URL).
- `alt`: Provides alternative text if the image doesn't load.
- `width` and `height`: Control the dimensions of the image.

6. HTML Links and Anchor Tags

Links are created using the `<a>` tag.

```
<a href="https://example.com">Visit Example Website</a>
```

- `href`: Specifies the URL to link to.
- Link text is clickable.

LAB TASK:

1: Create a simple HTML webpage that serves as a personal homepage. The webpage should include a main heading with your name, a short paragraph introducing yourself, and a bulleted list of three hobbies or favorite activities. Additionally, add a hyperlink to a website you like and an image that represents one of your hobbies. Save the file as `homepage.html`.

2: Design a basic personal portfolio website consisting of two HTML pages. The first page should include a welcome message, a brief introduction, and a list of your top three favorite books or movies. The second page should contain a small gallery with three images of your choice and a heading above each image to describe it. Save the pages as `index.html` and `gallery.html`, and store them in a folder named `PortfolioWebsite`.

Lab # 2

Client-side technologies: HTML-II

OBJECTIVE

To learn how to create and use HTML tables for structuring data, use tables for layout design, type emojis, and create basic HTML forms.

THEORY

1: HTML Tables

HTML tables are used to display data in a structured format using rows and columns.

Basic Structure of an HTML Table:

```
8   <body>
9   |   <table>
10  |   |   <tr>
11  |   |   |   <th>Heading 1</th>
12  |   |   |   <th>Heading 2</th>
13  |   |   |   </tr>
14  |   |   |   <tr>
15  |   |   |   |   <td>Data 1</td>
16  |   |   |   |   <td>Data 2</td>
17  |   |   |   </tr>
18  |   |   </table>
19  |
20  </body>
21  </html>
```

- `<table>`: Defines the table.
- `<tr>`: Defines a table row.
- `<th>`: Defines a table header (bold and centered by default).
- `<td>`: Defines table data (regular cell).

Example

```
8   <body>
9       <table border="1">
10          <tr>
11              <th>Name</th>
12              <th>Age</th>
13              <th>City</th>
14          </tr>
15          <tr>
16              <td>Ali</td>
17              <td>22</td>
18              <td>Karachi</td>
19          </tr>
20          <tr>
21              <td>Ayesha</td>
22              <td>20</td>
23              <td>Lahore</td>
24          </tr>
25      </table>
26
27
28 </body>
```

Using HTML Tables for Layout

Tables can also be used to structure a webpage layout. While CSS is preferred for layouts, tables provide a simple way to create grid-based designs.

Example Layout Using Tables:

```
8   <body>
9       <table border="1" width="100%">
10          <tr>
11              <td colspan="2" align="center"><b>Header</b></td>
12          </tr>
13          <tr>
14              <td width="20%"><b>Sidebar</b></td>
15              <td>Main Content</td>
16          </tr>
17          <tr>
18              <td colspan="2" align="center"><b>Footer</b></td>
19          </tr>
20      </table>
21
22
23 </body>
```

HTML Tables Code Challenge

Try creating a timetable for your weekly classes using an HTML table. Include the following:

- Days of the week as rows.
- Subjects and time slots as columns.
- Use the <thead>, <tbody>, and <tfoot> tags to separate different sections of the table.

How to Type Emojis

Emojis can be added to an HTML document using their Unicode or by directly copying them.

Example:

```
8   <body>
9   |   <p>I ❤️ HTML! 🌟</p>
10  <p>Unicode Example: 😀 (😄)</p>
11  </body>
12  </html>
```

HTML Forms

Forms are a crucial part of web development that allow users to input data, which can then be sent to a server for processing. They are used for various purposes, such as user registration, login, feedback collection, online orders, and surveys. An HTML form consists of various form elements like text fields, checkboxes, radio buttons, dropdown menus, and submit buttons.

Key Attributes of the <form> Tag

- action: Specifies the URL where the form data will be sent.
- method: Defines how the data is sent to the server. Common methods are:
- GET: Appends form data to the URL; used for simple requests.
- POST: Sends form data in the HTTP body; used for sensitive or large data.
- enctype: Specifies the encoding type for the form data (e.g., multipart/form-data for file uploads).

Example of a Complete HTML Form

```
9      <form action="/submit" method="POST">
10          <label for="firstname">First Name:</label>
11          <input type="text" id="firstname" name="firstname" required><br><br>
12
13          <label for="lastname">Last Name:</label>
14          <input type="text" id="lastname" name="lastname" required><br><br>
15
16          <label for="age">Age:</label>
17          <input type="number" id="age" name="age" min="1" max="100"><br><br>
18
19          <label for="gender">Gender:</label>
20          <input type="radio" id="male" name="gender" value="male"> Male
21          <input type="radio" id="female" name="gender" value="female"> Female<br><br>
22
23          <label for="hobbies">Hobbies:</label>
24          <input type="checkbox" name="hobby" value="sports"> Sports
25          <input type="checkbox" name="hobby" value="music"> Music<br><br>
26
27          <label for="country">Country:</label>
28          <select id="country" name="country">
29              <option value="pakistan">Pakistan</option>
30              <option value="india">India</option>
31          </select><br><br>
32
33          <label for="bio">Short Bio:</label><br>
34          <textarea id="bio" name="bio" rows="5" cols="30" placeholder="Tell us about yourself..."></textarea><br><br>
35
36          <input type="submit" value="Submit">
37      </form>
```

LAB TASK

Create a webpage that includes a timetable of your weekly classes using an HTML table. The table should have headings for each day of the week and rows for each time slot. Also, include a registration form below the timetable where users can enter their name, email, and select their class timing from a dropdown.

HOME TASK

Design a basic feedback form using HTML. The form should include fields for name, email, feedback message (textarea), and a rating system (radio buttons). Add a submit button at the end and save the file as feedback.html.

Lab # 3

Client-side technologies: CSS-I

OBJECTIVE

To understand the basics of CSS, its syntax, different types of stylesheets, and how to use selectors, classes, and IDs to style elements effectively.

THEORY

1: Introduction to CSS

CSS (Cascading Style Sheets) is used to control the style and layout of web pages. It separates content (HTML) from design (CSS), making it easier to maintain and update websites. CSS allows you to set properties like colors, fonts, margins, padding, and layouts.

2: The Anatomy of CSS Syntax

CSS follows a simple syntax:

- A CSS rule consists of a selector and a declaration block.
- The **selector** specifies which HTML element(s) the rule applies to.
- The **declaration block** contains one or more declarations inside curly braces { }.
- Each declaration includes a property and a value, separated by a colon : and ends with a semicolon ;.

Example Syntax:

```
css
selector {
    property: value;
    property: value;
}
```

3: Inline CSS, Internal CSS, and External CSS

1. **Inline CSS:** Styles are applied directly within an HTML element using the `style` attribute.

Example:

```
html
```



```
<p style="color: blue; font-size: 16px;">This is a blue  
paragraph.</p>
```

2. **Internal CSS:** Styles are written inside a `<style>` tag within the `<head>` section of the HTML document.

Example:

```
html  
<style>  
  p {  
    color: green;  
    font-size: 18px;  
  }  
</style>
```

3. **External CSS:** Styles are defined in a separate file and linked to the HTML document using a `<link>` tag.

Example:

- o style.css (External File):

```
css  
h1 {  
  color: red;  
  text-align: center;  
}
```

- o HTML File:

```
html  
  
<link rel="stylesheet" href="style.css">
```

4: CSS Selectors

Selectors are patterns used to select and style elements.

Basic Selectors:

- `*`: Universal selector; selects all elements.

```
css  
* {  
  margin: 0;  
  padding: 0;  
}
```

- `element`: Selects all elements of a specific type.

```
css
```

```
p {  
    color: gray;  
}
```

- **.class:** Selects all elements with a specific class.

```
css
```

```
.highlight {  
    background-color: yellow;  
}
```

- **#id:** Selects a specific element with an ID.

```
css
```

```
#main-header {  
    font-size: 24px;  
}
```

- **[attribute]:** Selects elements with a specific attribute.

```
css
```

```
input[type="text"] {  
    border: 1px solid black;  
}
```

5: Classes vs. IDs

- **Classes (class):** Used to apply styles to multiple elements.

Example:

Html

```
<p class="highlight">This paragraph is highlighted.</p>  
<p class="highlight">This one too!</p>
```

Css

```
.highlight {  
    background-color: yellow;  
}
```

- **IDs (id):** Used to apply styles to a single, unique element.

Example:**Html**

```
<h1 id="main-title">Welcome to CSS!</h1>
```

Css

```
#main-title {  
    color: navy;  
    font-size: 36px;  
}
```

LAB TASK

Design a homepage for a university that includes:

- A main heading styled using internal CSS to display the university name prominently.
- A navigation bar for links like "Home," "About Us," "Departments," and "Contact Us," styled using external CSS.
- A welcome paragraph styled using inline CSS to set a custom font size and background color.

HOME TASK

Create a personal portfolio webpage that includes:

- A main heading with your name, styled using an ID.
- A section for your hobbies, each with a subheading styled using classes.
- A list of your skills (ordered or unordered), with custom styles applied using an external stylesheet.

Lab # 4

Client-side technologies: CSS-II

OBJECTIVE

To explore advanced CSS concepts, including the Box Model, positioning, typography, and the role of divs in website styling.

THEORY

1: Favicons

Favicons are small icons associated with a webpage, displayed in the browser tab or bookmark. They enhance user experience and brand recognition.

To add a favicon to your webpage, use the following code inside the `<head>` section:

```
html
```

```
<link rel="icon" href="favicon.ico" type="image/x-icon">
```

2: Divs

The `<div>` element is a container used to group other HTML elements for styling and layout purposes. Divs are highly versatile and help structure a webpage into meaningful sections.

Example:

```
html
```

```
<div class="header">Header Section</div>
<div class="content">Main Content</div>
<div class="footer">Footer Section</div>
```

3: The Box Model of Website Styling

The CSS Box Model describes the structure of an HTML element. It consists of:

- **Content:** The inner text or HTML of the element.
- **Padding:** Space between the content and the border.
- **Border:** Surrounds the padding.
- **Margin:** Space between the border and other elements.

Example:

CSS

```
div {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 10px;  
}
```

4: CSS Display Property

The `display` property controls how elements are rendered on a page.

Common values include:

- `block`: The element takes up the entire width.
- `inline`: The element flows within a line.
- `flex`: Used for flexible layouts.

Example:

CSS

```
p {  
  display: inline;  
}  
div {  
  display: block;  
}
```

5: CSS Static, Relative, and Absolute Positioning

Positioning determines how elements are placed on a webpage:

- **Static:** Default positioning. Elements are placed in the normal flow of the document.
- **Relative:** Positioned relative to its normal position.
- **Absolute:** Positioned relative to the nearest positioned ancestor.

Example:

CSS

```
div.static {  
    position: static;  
}  
div.relative {  
    position: relative;  
    top: 10px;  
    left: 20px;  
}  
div.absolute {  
    position: absolute;  
    top: 50px;  
    left: 100px;  
}
```

6: CSS Typography

Typography focuses on styling text to improve readability and aesthetics. Common properties include:

- **Font-family:** Specifies the font.
 - **Font-size:** Sets the text size.
 - **Font-weight:** Adjusts the thickness of the font.
 - **Line-height:** Controls the spacing between lines of text.
- Example:

```
css  
  
p {  
    font-family: Arial, sans-serif;  
    font-size: 16px;  
    font-weight: bold;  
    line-height: 1.5;  
}
```

LAB TASK:

Design a **product landing page** that includes:

1. A favicon for the page.
2. A header, content section, and footer styled using **divs** and the **Box Model**.
3. Use **relative** positioning to adjust the content placement and **absolute** positioning for a floating "Contact Us" button.
4. Style the typography of the page (e.g., font family, size, and line spacing).

HOME TASK:

Design a homepage for a book store that includes:

1. A header section with the store name styled using CSS typography.
2. A section for featured books, divided into categories using divs.
3. Use the Box Model to add padding, borders, and margins to the book sections.
4. Include a "Buy Now" button for each book, styled using CSS positioning and typography.

Lab # 5

Client-side technologies: BOOTSTRAP

OBJECTIVE

To understand and implement Bootstrap, a popular front-end framework, for building responsive and modern web designs.

THEORY

What is Bootstrap?

Bootstrap is an open-source CSS framework designed to simplify web development. It provides pre-designed components like navigation bars, buttons, modals, and forms, along with a responsive grid system to create mobile-friendly layouts. It helps developers save time by reducing the need to write extensive CSS.

1: Installing Bootstrap

You can include Bootstrap in your project in two ways:

1. **Using CDN:** Add the following `<link>` to the `<head>` section of your HTML file:

```
html

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/b
ootstrap.min.css" rel="stylesheet">
```

2. **Downloading Bootstrap:**

Download the Bootstrap files from getbootstrap.com, and include them in your project directory.

2: Web Design - Wire Framing

Wireframing is the process of sketching the layout and structure of a website before coding. It helps visualize the placement of elements like headers, navigation bars, and content sections. Tools like Figma, Adobe XD, or even paper sketches are commonly used for wireframing.

3: The Bootstrap Navigation Bar

Bootstrap's navigation bar component provides a responsive and customizable way to create menus. Here's an example:

html

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">MySite</a>
    <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav">
        <li class="nav-item"><a class="nav-link"
href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link"
href="#">About</a></li>
        <li class="nav-item"><a class="nav-link"
href="#">Contact</a></li>
      </ul>
    </div>
  </div>
</nav>
```

4: Bootstrap Grid Layout System

The grid system is the foundation of Bootstrap layouts. It uses rows and columns to create responsive layouts. A row can have up to 12 columns, and you can specify how many columns an element should span.

Example:

html

```
<div class="container">
  <div class="row">
    <div class="col-md-6">Left Column</div>
    <div class="col-md-6">Right Column</div>
  </div>
</div>
```

5: Adding Grid Layouts to Our Website

Use the grid system to structure your website's sections, such as headers, content areas, and footers, while ensuring they adapt to various screen sizes.

6: Bootstrap Containers

Bootstrap provides two types of containers to manage layouts:

1. **.container:** Fixed width and responsive.
2. **.container-fluid:** Full width across all screen sizes.

Example:

html

```
<div class="container">
  <h1>Fixed Container</h1>
</div>
<div class="container-fluid">
  <h1>Fluid Container</h1>
</div>
```

7: Bootstrap Buttons & Font Awesome

Bootstrap includes pre-styled buttons, which can be customized using classes like `.btn-primary`, `.btn-success`, and `.btn-danger`. Font Awesome is an icon library often used alongside Bootstrap for adding icons.

Example:

html

```
<button class="btn btn-primary">Click Me</button>
<i class="fa fa-check"></i>
```

LAB TASK

Create a responsive webpage for a **Tech Startup**, including:

1. A **navigation bar** with links to Home, Services, and Contact Us.
2. A **hero section** using the **Bootstrap grid system** with a heading, subheading, and a "Learn More" button styled using Bootstrap buttons.
3. Use a **container-fluid** to include a list of services, each in its own column.

HOME TASK

Design a responsive webpage for a **Travel Agency**, including:

1. A **navigation bar** with a logo and links to Home, Packages, and Contact Us.
2. Use the **grid layout system** to display three featured travel destinations with images and descriptions.
3. Add **Font Awesome icons** for contact information (e.g., phone, email).

Lab # 6

Client-side technologies: Javascript - I

Objective

To learn the fundamentals of JavaScript, including its data types, variables, control statements, and functions, and how it adds dynamic behavior to websites.

Theory

JavaScript is a powerful client-side scripting language that enables interactive and dynamic features on web pages. It is widely used for tasks such as form validation, creating animations, and handling user events. JavaScript can be embedded within an HTML document or included as an external file.

JavaScript Alerts - Adding Behavior to Websites

JavaScript alerts are used to display messages to users. The `alert()` function is often used for debugging or providing basic notifications to users.

Example:

```
javascript  
  
alert("Welcome to JavaScript!");
```

Data Types

JavaScript has several data types, including:

- **String:** Text, e.g., "Hello, world!"
- **Number:** Numerical values, e.g., 42 or 3.14
- **Boolean:** Logical values, true or false
- **Object:** Collection of properties, e.g., { name: "John", age: 25 }
- **Array:** List of values, e.g., [1, 2, 3]
- **Null and Undefined:** Special data types representing empty or uninitialized values.

JavaScript Variables

Variables store data values. Use `var`, `let`, or `const` to declare variables.

Example:

```
javascript

let name = "Alice"; // A variable that can change
const pi = 3.14;    // A constant value
```

String Functions

JavaScript provides many functions to manipulate strings, such as:

- `length`: Returns the length of a string.
- `toUpperCase()`: Converts a string to uppercase.
- `toLowerCase()`: Converts a string to lowercase.

Example:

```
javascript

let text = "JavaScript";
console.log(text.length);           // Output: 10
console.log(text.toUpperCase());    // Output: "JAVASCRIPT"
```

Basic Arithmetic and the Modulo Operator

JavaScript supports arithmetic operators like `+`, `-`, `*`, `/`, and `%` (modulo).

Example:

```
javascript

let a = 10, b = 3;
console.log(a % b); // Output: 1 (remainder of 10 / 3)
```

Increment and Decrement Expressions

Increment (`++`) and decrement (`--`) operators increase or decrease a value by 1.

Example:

```
javascript

let x = 5;
x++; // x becomes 6
x--; // x becomes 5
```

Control Statements: Using If-Else Conditionals

if-else statements control program flow based on conditions.

Example:

```
javascript

let score = 85;
if (score > 50) {
  console.log("You passed!");
} else {
  console.log("You failed!");
}
```

Control Statements: While Loops

A while loop runs as long as a condition is true.

Example:

```
javascript

let count = 1;
while (count <= 5) {
  console.log(count); // Outputs 1, 2, 3, 4, 5
  count++;
}
```

Control Statements: For Loops

A for loop is ideal for iterating a specific number of times.

Example:

```
javascript

for (let i = 1; i <= 5; i++) {
  console.log(i); // Outputs 1, 2, 3, 4, 5
}
```

JavaScript Objects and Functions

Objects in JavaScript are collections of properties and methods. Functions define reusable blocks of code.

Example of an object and a function:

```
javascript

let student = {
  name: "John",
  age: 22,
  greet: function () {
    console.log(`Hello, my name is ${this.name}.`);
  }
};
student.greet(); // Output: "Hello, my name is John."

function add(a, b) {
  return a + b;
}
console.log(add(5, 10)); // Output: 15
```

Lab Task

Design a webpage using JavaScript to perform the following:

1. Display an alert message welcoming users to your webpage.
2. Create a form to input a user's name and display a personalized greeting using JavaScript.
3. Implement a `for` loop to display the first 10 natural numbers in the console.

Home Task

Create a simple JavaScript program to:

1. Ask the user for a number input using `prompt()`.
2. Check whether the number is even or odd using an `if-else` statement.
3. Display the result in an alert box.

Lab # 7

Client-side technologies: Javascript - II

Objective

To understand how JavaScript interacts with HTML and CSS using the Document Object Model (DOM), and how to manipulate webpage elements dynamically.

Theory

The **Document Object Model (DOM)** is a programming interface for web documents. It represents the structure of a web page as a tree of objects, where each node represents a part of the page (e.g., elements, attributes, text). JavaScript can interact with the DOM to manipulate HTML and CSS elements dynamically, allowing for interactive web pages.

In this lab, you will learn how JavaScript can:

- Access HTML elements using the DOM.
- Modify elements dynamically.
- Change CSS styles via JavaScript.
- Respond to user actions by manipulating the DOM.

What is the DOM?

The DOM is essentially a representation of the HTML document as a tree structure. Each element in the HTML document is an object in the tree, and JavaScript can access and modify these elements through the DOM.

For example, in the HTML structure:

html

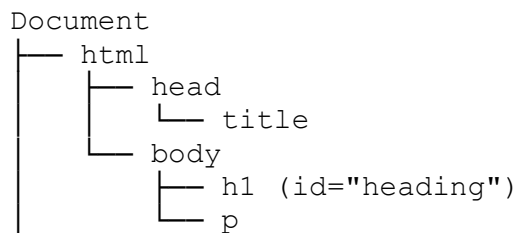
```
<!DOCTYPE html>
<html>
  <head>
    <title>DOM Example</title>
  </head>
  <body>
    <h1 id="heading">Hello, DOM!</h1>
    <p>Welcome to learning JavaScript and DOM.</p>
  </body>
```



```
</html>
```

The DOM representation might look like this:

```
less
```



Interacting with the DOM Using JavaScript

Accessing DOM Elements

JavaScript can access HTML elements using methods like:

- `getElementById()`: Access elements by their `id`.
- `getElementsByTagName()`: Access elements by their tag name.
- `getElementsByClassName()`: Access elements by their class name.
- `querySelector()`: Select the first element matching a CSS selector.
- `querySelectorAll()`: Select all elements matching a CSS selector.

Example: Accessing Elements

```
javascript
```

```
// Access an element by its ID
let heading = document.getElementById('heading');
console.log(heading); // Output: <h1 id="heading">Hello, DOM!</h1>

// Access all <p> elements by tag name
let paragraphs = document.getElementsByTagName('p');
console.log(paragraphs); // Output: [<p>Welcome to learning JavaScript
and DOM.</p>]
```

Manipulating DOM Elements

Once you access an element, you can manipulate its content, attributes, or style.

Changing Content

To change the content of an element, you can modify the `innerHTML` property.

```
javascript
```

```
// Change the text content of the heading
```

```
heading.innerHTML = "JavaScript DOM Manipulation";
```

Changing Styles

You can change the CSS styles of an element by modifying its `style` property.

```
javascript

// Change the color and font size of the heading
heading.style.color = "blue";
heading.style.fontSize = "30px";
```

Changing Attributes

To modify attributes like `src`, `href`, or `class`, you can use the `setAttribute()` method.

```
javascript

// Change the "href" attribute of a link
let link = document.getElementById('myLink');
link.setAttribute("href", "https://www.example.com");
```

Creating New Elements

You can also create new elements and append them to the document.

```
javascript

// Create a new paragraph
let newParagraph = document.createElement("p");
newParagraph.innerHTML = "This is a new paragraph created by
JavaScript.";

// Append the new paragraph to the body
document.body.appendChild(newParagraph);
```

Removing Elements

To remove an element, you can use the `removeChild()` method.

```
javascript

// Remove the first paragraph
document.body.removeChild(paragraphs[0]);
```

Event Handling

JavaScript allows you to respond to user interactions using events like `click`, `mouseover`, `keydown`, etc. You can attach event listeners to DOM elements.

```
javascript
```

```
// Add a click event to the heading
heading.addEventListener("click", function() {
  alert("Heading clicked!");
});
```

DOM Example: Interactive Webpage

Here's a full example that demonstrates how to use JavaScript with HTML and CSS through the DOM:

HTML:

```
<body>
  <h1 id="heading">Click the Button to Change Content</h1>
  <p id="text">This paragraph will be updated by JavaScript.</p>
  <button id="myButton">Click Me</button>
</body>
```

CSS:

```
<style>
  #myButton {
    padding: 10px 20px;
    background-color: lightblue;
    border: none;
    cursor: pointer;
  }

  #myButton:hover {
    background-color: lightgreen;
  }
</style>
```

JAVASCRIPT:

```
<script>
  // Accessing elements
  let heading = document.getElementById("heading");
  let paragraph = document.getElementById("text");
  let button = document.getElementById("myButton");

  // Change content when button is clicked
  button.addEventListener("click", function() {
    heading.innerHTML = "Content Changed!";
    paragraph.innerHTML = "You have successfully clicked the button and changed the content.";
    paragraph.style.color = "green";
  });
</script>
```

OUTPUT (BEFORE CLICKING):

Click the Button to Change Content

This paragraph will be updated by JavaScript.

Click Me

OUTPUT (AFTER CLICKING):

Content Changed!

You have successfully clicked the button and changed the content.

Click Me

Lab Task:

1. Create a webpage with a heading, a paragraph, and a button.
2. Use JavaScript to change the content of the paragraph when the button is clicked.
3. Modify the button's style using JavaScript when the mouse hovers over it (using `mouseover` and `mouseout` events).
4. Add a new paragraph element dynamically when the user clicks the button.

Home Task:

1. Design a webpage with at least three sections: a header, content area, and footer.
2. Use JavaScript to dynamically update the header text when a button is clicked.
3. Allow the user to change the background color of the content area by selecting a color from a dropdown menu.
4. Use JavaScript to add a new footer element dynamically after 5 seconds of loading the page.

Lab # 8

Client-side technologies: jQuery and JSON

OBJECTIVE

To understand the fundamentals of jQuery and JSON and how they are used together in modern web development for creating interactive and dynamic web applications.

THEORY

What is jQuery?

jQuery is a fast, small, and feature-rich JavaScript library. It simplifies things like DOM manipulation, event handling, and AJAX requests with a concise and readable syntax. Developers widely use jQuery to create dynamic and interactive web applications.

- **Features of jQuery:**
 - Simplifies DOM manipulation.
 - Handles cross-browser compatibility.
 - Provides an easy-to-use API for AJAX.
 - Supports event handling.
 - Animations and effects for enhancing UI.

Example: Basic jQuery Syntax

The jQuery syntax is simple and easy to use:

```
javascript

$(document).ready(function() {
    // This code runs when the DOM is fully loaded
    $("p").click(function() {
        $(this).css("color", "blue");
    });
});
```

In this example:

- `$` is used to select elements.
- The `.click()` method is an event handler.
- `.css()` is used to style the element.

What is JSON (JavaScript Object Notation)?

JSON is a lightweight data format used for storing and exchanging data. It is easy for humans to read and write and easy for machines to parse and generate.

- **Features of JSON:**
 - Based on key-value pairs.
 - Easy to integrate with most programming languages.
 - Used in APIs for data exchange.

Example: JSON Data Structure

```
json
{
  "name": "John Doe",
  "age": 30,
  "isStudent": false,
  "skills": ["HTML", "CSS", "JavaScript"]
}
```

Topics in Detail

1. DOM Manipulation with jQuery

jQuery makes it easier to interact with the Document Object Model (DOM). You can add, remove, or change HTML elements dynamically.

Example: Add a New Paragraph

```
javascript

$(document).ready(function() {
  $("#addBtn").click(function() {
    $("#content").append("<p>This is a new
paragraph!</p>");
  });
});
```

```
});});
```

HTML Example:

```
html
```

```
<div id="content">
  <p>Existing content.</p>
</div>
<button id="addBtn">Add Paragraph</button>
```

2. Event Handling in jQuery

With jQuery, you can bind events to HTML elements and define actions.

Example: Hover Effect

```
javascript
```

```
$(document).ready(function() {
  $("button").hover(function() {
    $(this).css("background-color", "green");
  }, function() {
    $(this).css("background-color", "");
  });
});
```

HTML Example:

```
html
```

```
<button>Hover over me!</button>
```

3. AJAX Requests with jQuery

jQuery simplifies AJAX by providing easy-to-use methods for sending and receiving data from a server without requiring a page reload. It supports common operations like GET and POST requests with functions such as `$.get()`, `$.post()`, and `$.ajax()`. These methods allow seamless data exchange, making web applications faster and more dynamic. jQuery also includes built-in error handling and options to work with different data formats, like JSON or XML.

Example: Fetch Data from a Server

javascript

```
$(document).ready(function() {  
    $("#fetchData").click(function() {  
        $.get("https://api.example.com/data", function(response) {  
            $("#result").html(response.name);  
        });  
    });  
});
```

HTML Example:

html

```
<button id="fetchData">Get Data</button>  
<div id="result"></div>
```

4. JSON and jQuery

JSON works seamlessly with jQuery to transfer data between the client and the server.

Example: Display JSON Data in HTML

javascript

```
$(document).ready(function() {  
    const jsonData = {  
        "name": "Alice",  
        "age": 25,  
        "skills": ["HTML", "CSS", "JavaScript"]  
    };  
  
    $("#showData").click(function() {  
        $("#output").html(`  
            <p>Name: ${jsonData.name}</p>  
            <p>Age: ${jsonData.age}</p>  
            <p>Skills: ${jsonData.skills.join(", ")}</p>  
        `);  
    });  
});
```


HTML Example:

html

```
<button id="showData">Show Data</button>
<div id="output"></div>
```

5. Animations with jQuery

jQuery provides built-in methods to create smooth animations.

Example: Slide Toggle

javascript

```
$(document).ready(function() {
    $("#toggleBtn").click(function() {
        $("#box").slideToggle();
    });
});
```

HTML Example:

html

```
<button id="toggleBtn">Toggle Box</button>
<div id="box" style="width: 100px; height: 100px; background-
color: red;
```

Display and Toggle JSON Data with jQuery:

This example combines **JSON**, **DOM manipulation**, **event handling**, and **animations** to demonstrate their usage in a single scenario.

JavaScript and jQuery Code:

javascript

```
$(document).ready(function () {
    // JSON Data
    const students = [
        { name: "John Doe", age: 22, skills: ["HTML", "CSS",
"JavaScript"] },
        { name: "Jane Smith", age: 24, skills: ["Python", "Django",
"React"] },
```

```

        { name: "Alice Johnson", age: 21, skills: ["C++", "Java",
"Spring"] }
    ];

    // Display students on button click
    $("#showStudents").click(function () {
        let output = "<h3>Student List</h3><ul>";
        students.forEach(student => {
            output += `<li>${student.name}, Age: ${student.age},
Skills: ${student.skills.join(", ")}</li>`;
        });
        output += "</ul>";
        $("#studentData").html(output).fadeIn(); // Display data with
fade-in animation
    });

    // Hide student data
    $("#hideStudents").click(function () {
        $("#studentData").fadeOut(); // Hide data with fade-out
animation
    });
});

```

HTML Structure:

```

html

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>jQuery and JSON Example</title>
    <script src="https://code.jquery.com/jquery-
3.6.0.min.js"></script>
    <style>
        #studentData {
            display: none;
            background-color: #f9f9f9;
            padding: 10px;
            border: 1px solid #ccc;
            border-radius: 5px;
        }
        button {
            margin: 5px;
            padding: 10px;
            background-color: #007bff;
            color: #fff;

```

```
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }
    button:hover {
        background-color: #0056b3;
    }
</style>
</head>
<body>
    <h1>Interactive Student List</h1>
    <button id="showStudents">Show Students</button>
    <button id="hideStudents">Hide Students</button>
    <div id="studentData"></div>
    <script src="java1.js"></script>
</body>
</html>
```

OUTPUT:

Interactive Student List

[Show Students](#)[Hide Students](#)

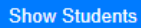
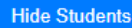
AFTER CLICKING SHOW STUDENTS:

Interactive Student List

[Show Students](#)[Hide Students](#)

Student List

- John Doe, Age: 22, Skills: HTML, CSS, JavaScript
- Jane Smith, Age: 24, Skills: Python, Django, React
- Alice Johnson, Age: 21, Skills: C++, Java, Spring

AFTER CLICKING HIDE STUDENTS:**Interactive Student List**A blue rectangular button with the text "Show Students" in white.A blue rectangular button with the text "Hide Students" in white.**LAB TASK**

Create a webpage that uses jQuery and JSON to implement the following:

- Fetch and display a list of students (name, age, and grade) from a JSON object using jQuery.
- Add a button that toggles the visibility of the entire student details section using fade-in and fade-out animations.
- Include a search field where users can type and dynamically filter student names, showing only matching results in real time.

HOME TASK

Design an interactive product catalog webpage with the following features:

- Use a JSON object to fetch product data, including name, price, description, and image URLs, and display them in a grid layout.
- Implement a category filter using checkboxes that dynamically updates the displayed products based on the selected category (e.g., Electronics, Clothing).
- For each product, include a "Details" button that reveals additional information (e.g., specifications, availability) using a slide animation.

Lab # 9

Client-side technologies: INTRODUCTION TO REACT JS

OBJECTIVE

To understand the fundamentals of React JS, including its core concepts, components, and state management, and to build dynamic and interactive user interfaces with React JS.

THEORY

What is React JS?

React JS is a popular **JavaScript library** developed by **Facebook** for building user interfaces, especially single-page applications (SPAs). React allows developers to create reusable UI components, making the code modular, maintainable, and efficient.

Key Features of React JS:

1. **Component-Based Architecture** – Applications are broken into smaller, reusable components.
2. **Virtual DOM** – React creates a lightweight copy of the real DOM to optimize rendering and improve performance.
3. **Unidirectional Data Flow** – Data flows in a single direction for better debugging and predictable behavior.
4. **JSX (JavaScript XML)** – A syntax extension that allows mixing HTML and JavaScript.
5. **Declarative UI** – React focuses on "what to render" rather than "how to render."

Setting Up React Environment

To start with React, you need to install **Node.js** and **npm** (Node Package Manager).

1. **Install Node.js and npm**
Download and install Node.js from <https://nodejs.org/>.
2. **Install React using Create React App** Run the following command in your terminal:

```
npx create-react-app my-app
cd my-app
npm start
```

- `npx create-react-app` generates a React application with all the necessary configurations.
- `npm start` starts the development server at `http://localhost:3000`.

React Basics

1. JSX (JavaScript XML)

JSX allows you to write HTML-like syntax within JavaScript. JSX makes React code more readable.

Example: Using JSX

```
javascript

import React from 'react';
import ReactDOM from 'react-dom';

const element = <h1>Hello, React!</h1>;
ReactDOM.render(element, document.getElementById('root'));
```

Here:

- `<h1>` is HTML embedded inside JavaScript using JSX.
- `ReactDOM.render` renders the content inside the specified DOM element.

2. Components in React

A **component** is the building block of a React application. React has two types of components:

1. **Functional Components** – Written as JavaScript functions.
2. **Class Components** – Written using ES6 classes (older approach).

Example: Functional Component

```
javascript

function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}

export default Welcome;
```

Here:

- `props` stands for "properties" and is used to pass data into components.

3. Rendering Components

Components are rendered into the DOM using `ReactDOM.render`.

Example: Rendering a Component

```
javascript

import React from 'react';
import ReactDOM from 'react-dom';
import Welcome from './Welcome';

ReactDOM.render(<Welcome name="Alice" />,
  document.getElementById('root'));
```

Output:

```
Hello, Alice!
```

4. State and Hooks

React components use **state** to hold dynamic data. Functional components use **Hooks** to manage state.

Example: Using the `useState` Hook

```
javascript

import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count +
1)}>Increment</button>
    </div>
  );
}

export default Counter;
```

Here:

- `useState(0)` initializes `count` to 0.
- `setCount` updates the value of `count`.

5. Event Handling

React uses camelCase for event handlers and the `on` prefix (e.g., `onClick`, `onChange`).

Example: Handling Events

```
javascript

function ButtonClick() {
  function handleClick() {
    alert('Button clicked!');
  }

  return <button onClick={handleClick}>Click Me</button>;
}

export default ButtonClick;
```


1. Components and Props

- Create functional components.
- Use props to pass data between components.

2. State Management

- Use `useState` for managing state in functional components.

3. Rendering Lists

- Use `.map()` to render lists dynamically.

4. Event Handling

- Bind event handlers to HTML elements in React.

5. React Lifecycle (for class components)

- `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`.

EXAMPLES

1. Rendering a List of Items

javascript

```
function StudentList() {
  const students = ['John', 'Jane', 'Alice'];

  return (
    <ul>
      {students.map((student, index) => (
        <li key={index}>{student}</li>
      ))}
    </ul>
  );
}
```

```
export default StudentList;
```

2. Simple Counter App

javascript

```
import React, { useState } from 'react';

function CounterApp() {
  const [count, setCount] = useState(0);

  return (
    <div>
```

```
        <h1>Counter: {count}</h1>
        <button onClick={() => setCount(count +
1)}}>Increase</button>
        <button onClick={() => setCount(count -
1)}}>Decrease</button>
      </div>
    );
  }

export default CounterApp;
```

LAB TASK

Task 1: Build a Student Information App

1. Create a React app that displays a list of student names and ages using a functional component.
2. Use `useState` to add a "Show More" button that displays additional information (e.g., skills).
3. Render the list dynamically using the `map()` function.

HOME TASK

Task 2: Create a Product Catalog

1. Fetch product data from a JSON array.
2. Display the product name, price, and description in a grid layout.
3. Add a "Show More" button for each product that expands to display additional details (like reviews).
4. Use `useState` for dynamic state updates.

Example JSON Data:

```
const products = [
  { name: 'Laptop', price: '$1000', description: 'High-performance laptop' },
  { name: 'Phone', price: '$500', description: 'Smartphone with a great camera' }
];
```

Lab # 10

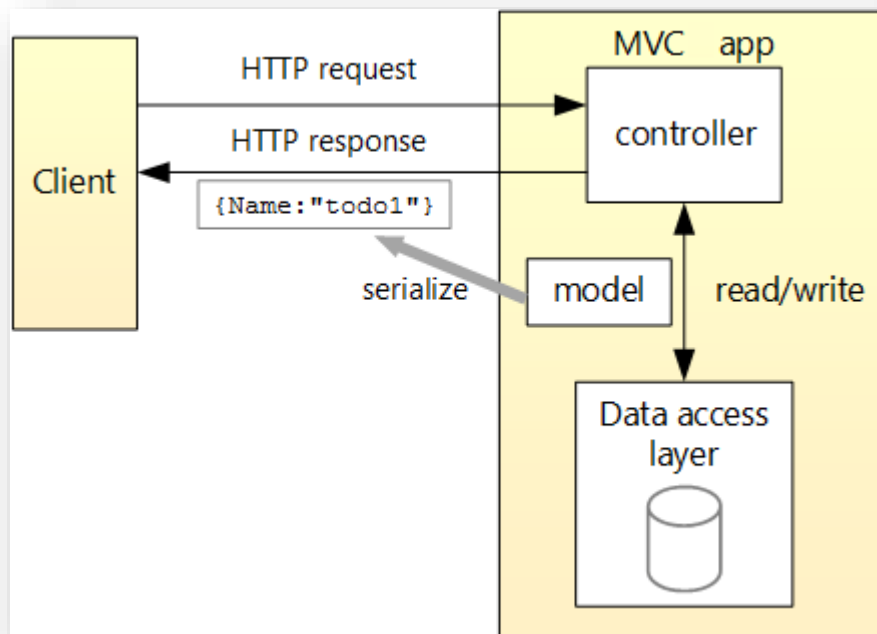
Server-side technologies: C#, WebAPI Basics

Objective:

- To understand the basics of server-side technologies using C#.
- To create and test a simple WebAPI using .NET framework.

Theory Background:

Server-side programming is a crucial aspect of modern web development. It focuses on developing server-based applications or services, maintaining data, processing requests, and reacting to client-side programs such as desktop software, mobile apps, and web browsers. For tasks like collecting data from a database, carrying out business logic, or carrying out intricate calculations, these services facilitate smooth communication between the user and the server.



Microsoft created C#, a strong and flexible programming language with many server-side programming features. C# is a great tool for creating scalable, secure, and effective web services when used with the .NET platform. High performance and dependability are ensured by the organized environment that the .NET framework offers for writing, testing, and deploying server-side applications.

ASP.NET WebAPI is one of the most widely used tools in this ecosystem. A framework for creating HTTP-based services is called a WebAPI. Various client types, such as desktop programs, mobile devices, and online apps, can use these services. Developers can establish endpoints (URLs) that operate as gateways for client-server interaction using WebAPIs. For example:

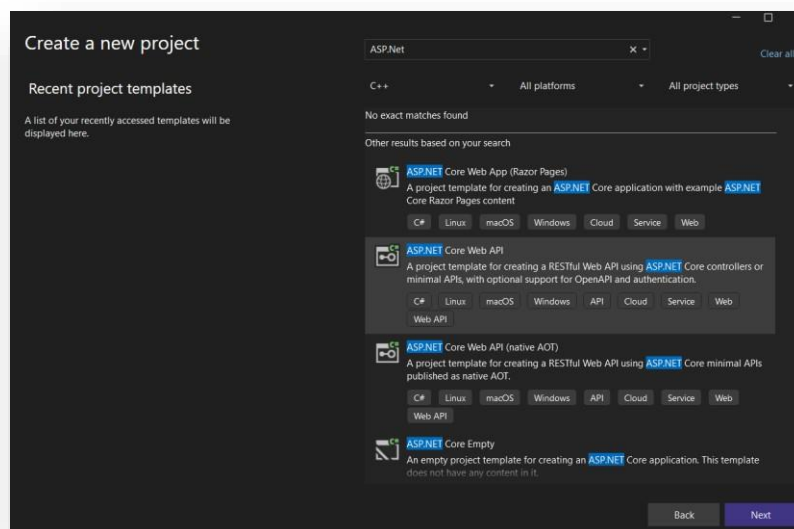
- A web application can request data from a database.
- A mobile app can send data to update user profiles.
- A desktop application can query the API for real-time information.

RESTful (Representational State Transfer) principles, which emphasize scalability, simplicity, and statelessness, are adhered to by the WebAPI framework. Because of this architecture, the APIs are guaranteed to be user-friendly, adaptable, and interoperable with a variety of platforms and programming languages. Developers can design reliable services that support dynamic and interactive applications in a range of contexts by utilizing C# and ASP.NET WebAPI.

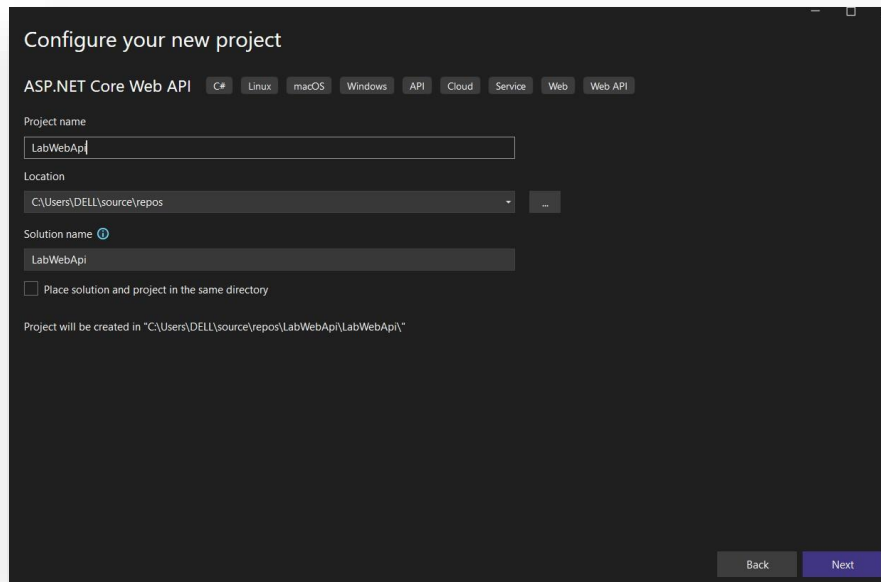
Experiment Steps:

Part A: Setting Up the Environment

1. Install Visual Studio (if not already installed).
2. Create a new project:
 - Open Visual Studio and click on **Create a new project**.
 - Select **ASP.NET Core Web API** and click **Next**.

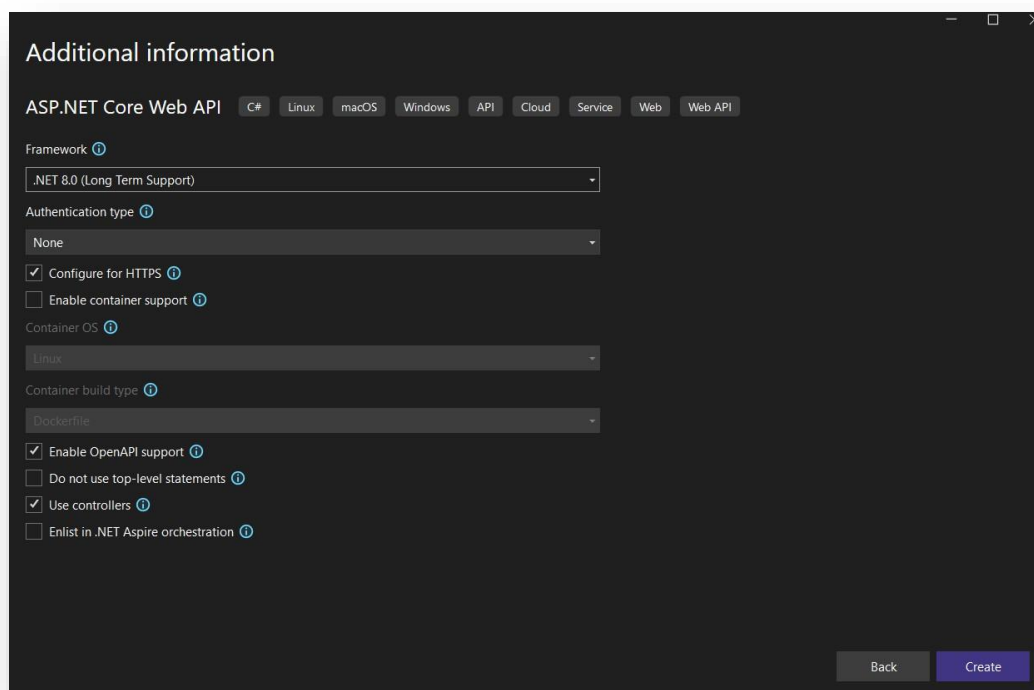


- Name your project (e.g., "LabWebAPI") and click **Create**.



The screenshot shows the 'Configure your new project' window. At the top, it says 'Configure your new project'. Below that, there are tabs for 'ASP.NET Core Web API', 'C#', 'Linux', 'macOS', 'Windows', 'API', 'Cloud', 'Service', 'Web', and 'Web API'. The 'Project name' field contains 'LabWebApi'. The 'Location' field shows 'C:\Users\DELL\source\repos'. The 'Solution name' field also contains 'LabWebApi'. There is a checkbox for 'Place solution and project in the same directory' which is unchecked. At the bottom, it says 'Project will be created in "C:\Users\DELL\source\repos\LabWebApi\LabWebApi"'. There are 'Back' and 'Next' buttons at the bottom right.

3. Choose the **.NET 8.0** framework (or the latest version available) and click **Create**.

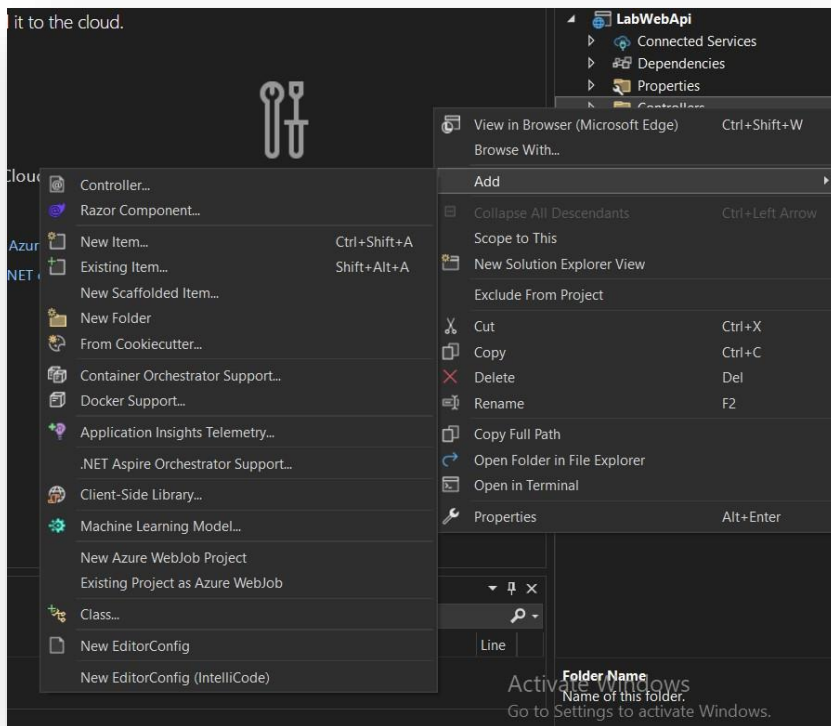


The screenshot shows the 'Additional information' window. At the top, it says 'Additional information'. Below that, there are tabs for 'ASP.NET Core Web API', 'C#', 'Linux', 'macOS', 'Windows', 'API', 'Cloud', 'Service', 'Web', and 'Web API'. The 'Framework' dropdown is set to '.NET 8.0 (Long Term Support)'. The 'Authentication type' dropdown is set to 'None'. There are checkboxes for 'Configure for HTTPS' (checked), 'Enable container support' (unchecked), 'Container OS' (set to 'Linux'), 'Container build type' (set to 'Dockerfile'), 'Enable OpenAPI support' (checked), 'Do not use top-level statements' (unchecked), 'Use controllers' (checked), and 'Enlist in .NET Aspire orchestration' (unchecked). There are 'Back' and 'Create' buttons at the bottom right.

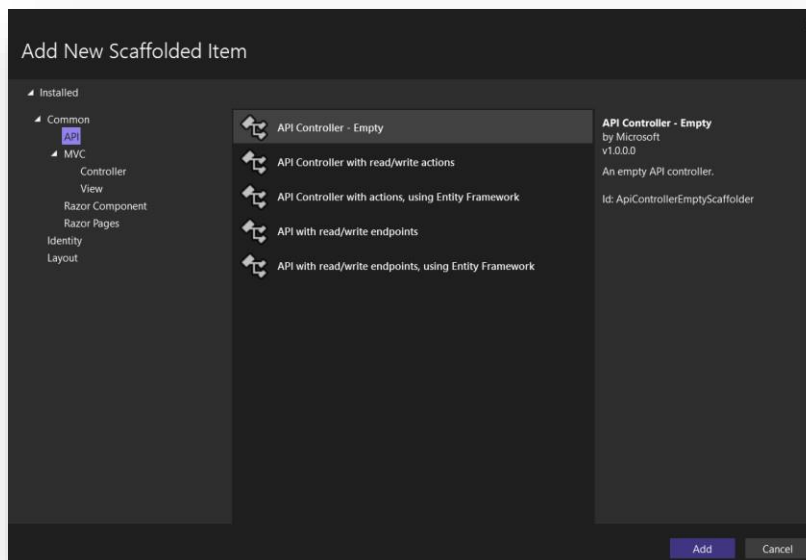
Part B: Creating a Simple WebAPI

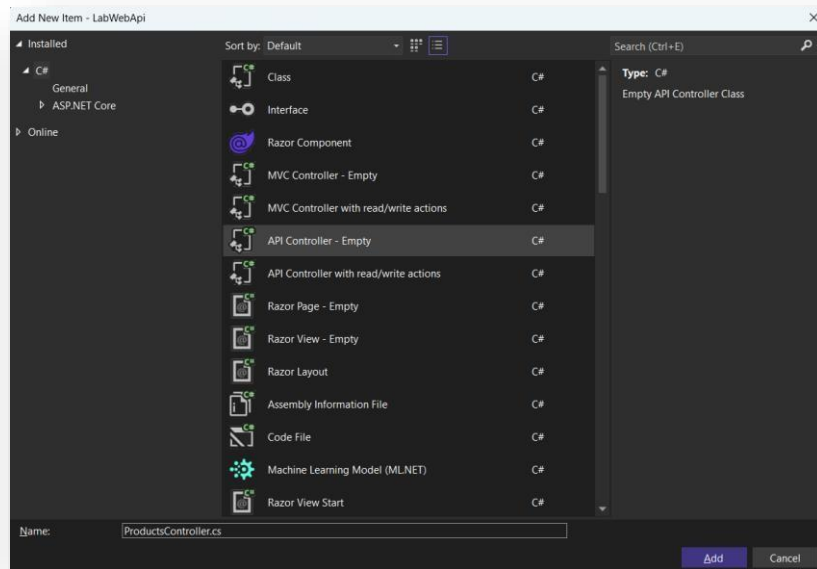
1. Create a Controller:

- In the **Solution Explorer**, right-click the **Controllers** folder and select **Add > Controller**.

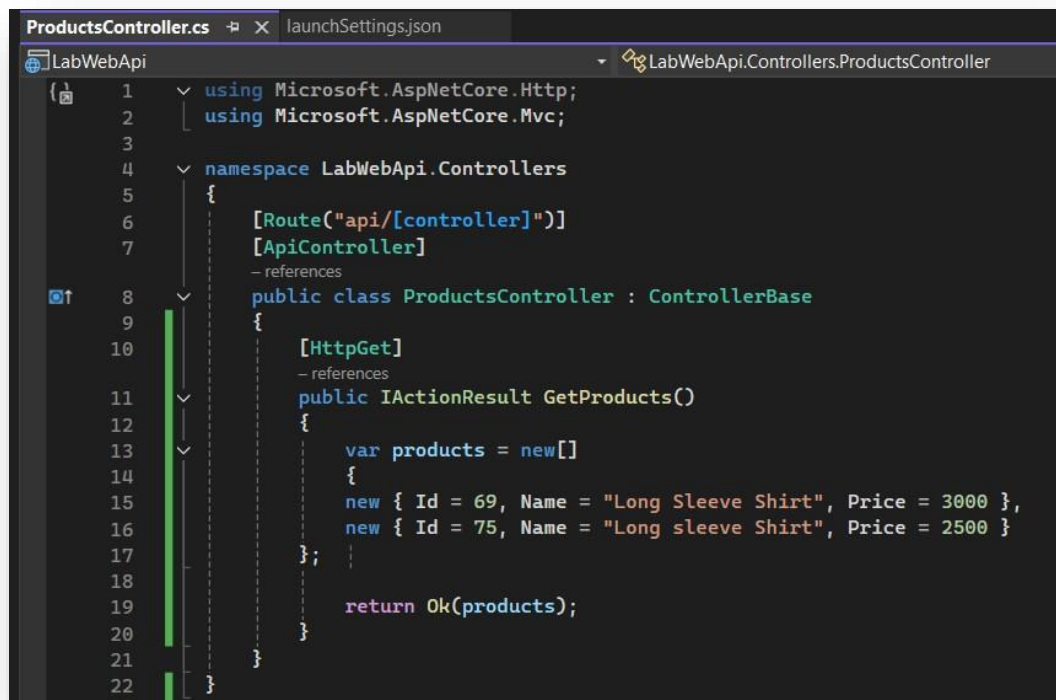


- Choose **API Controller - Empty** and name it “ProductsController.cs”.





- write the following code to define the API:



2. Run the Application:

- Press **Ctrl + F5** to run the application.

Note: Option 01**Resolving "Your connection isn't private" Error**

If you encounter the "**Your connection isn't private**" error, follow these steps:

1. Run your application in Visual Studio as usual.
2. Open the **Output** window in Visual Studio to find the **HTTP URL** of your application (e.g., `http://localhost:<port>`).
3. Instead of using the HTTPS version (`https://localhost:<port>`), use the HTTP URL (`http://localhost:<port>`).
4. Append `/swagger` to the URL (e.g., `http://localhost:<port>/swagger`) to run the application.

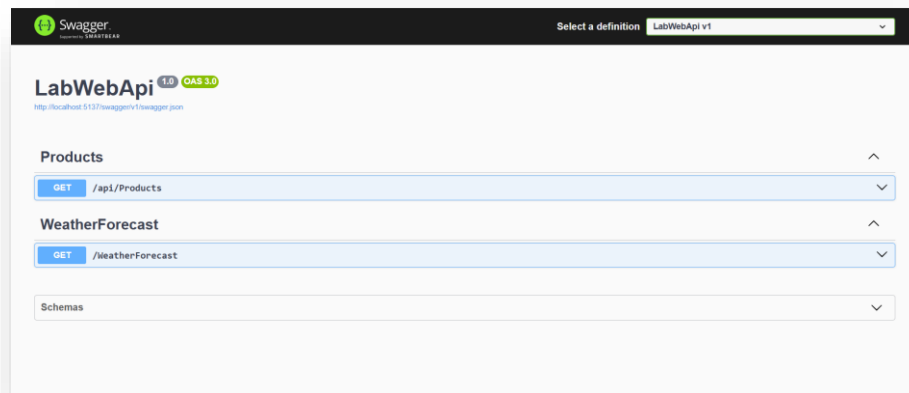
Or Option 02**Temporarily Disable HTTPS by Commenting It Out**

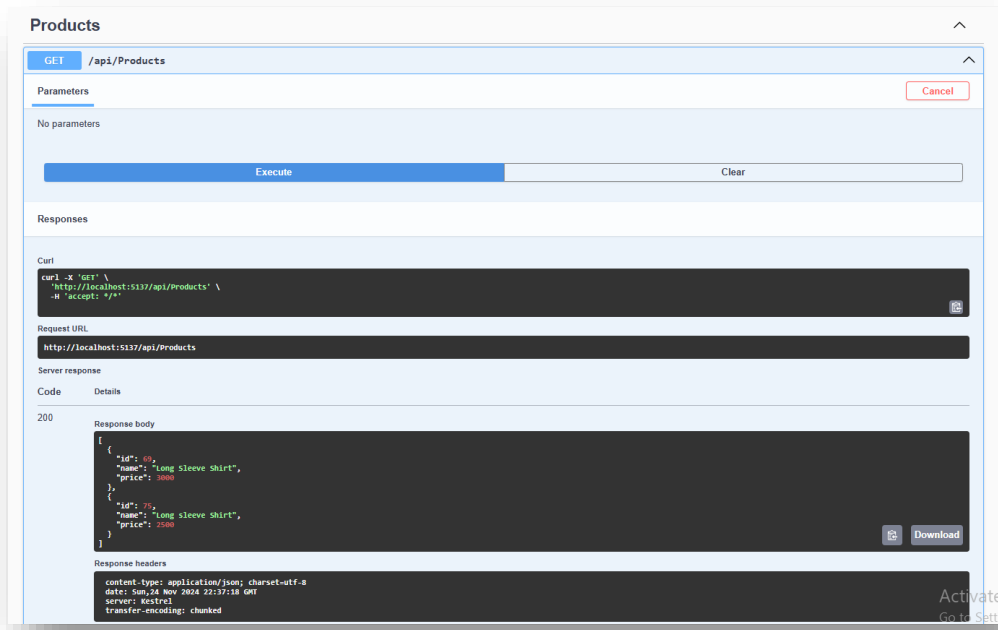
If HTTPS is not required for your application temporarily, follow these steps:

1. Open the `launchSettings.json` file located in the **Properties** folder of your project.
2. Locate the `profiles` section in the JSON file.
3. Find your application's profile (e.g., **IIS Express** or `<ProjectName>`).
4. Temporarily comment out the entire HTTPS part.
5. Save the file and re-run the application.

This change ensures that your application runs on HTTP instead of HTTPS. Once you need HTTPS again, simply uncomment the HTTPS URL and restart the application.

- The browser will open the API documentation using Swagger (if enabled).





➤ But What is Swagger?

Swagger is a set of open-source tools built around the **OpenAPI Specification (OAS)** that helps developers design, build, document, and consume RESTful APIs. It enables APIs to describe their structure in a machine-readable format, forming the foundation of its functionality.

Key Features and Components

1. Swagger Editor:

A browser-based tool for writing and editing API documentation using OpenAPI specifications. It supports both YAML and JSON formats, providing a side-by-side view of the code and the resulting documentation.

2. Swagger UI:

A tool that generates interactive API documentation from OpenAPI specifications. It allows users to try out API calls directly in the browser, making API testing and understanding easier.

3. Swagger Codegen:

A tool that generates server stubs, client SDKs, and libraries from an OpenAPI specification. It streamlines the process of setting up server logic and client-side code for APIs.

How Swagger Works

Swagger works by requesting your API to return a **YAML** or **JSON** file containing a detailed description of the entire API. This file adheres to the OpenAPI

Specification and includes:

- Supported operations.
- Parameters and return types.
- Authorization methods.
- Contact information and terms of use.

Benefits of Using Swagger

1. **Interactive Documentation:**

Swagger UI offers a user-friendly interface for API documentation, allowing users to interact with the API directly from the documentation.

2. **Automated Code Generation:**

Swagger Codegen enables the automatic generation of client libraries, server stubs, and API documentation, saving time and reducing errors.

3. **Standardization:**

By following the OpenAPI Specification, Swagger ensures that APIs are described consistently and in a standard format, making them easier to understand and use.

Practical Applications

1. **API Design:**

Use Swagger Editor to design APIs before implementation, ensuring all necessary details are included.

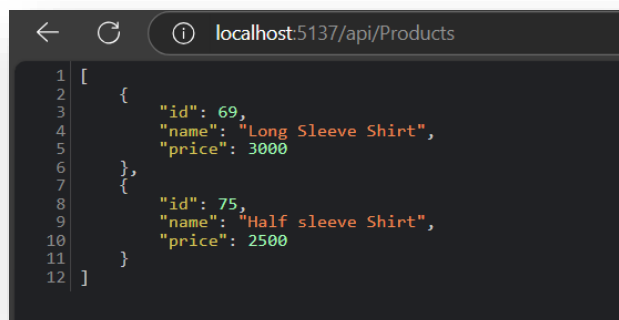
2. **Client Libraries:**

Generate client libraries in multiple languages using Swagger Codegen, simplifying integration for developers.

3. **Testing:**

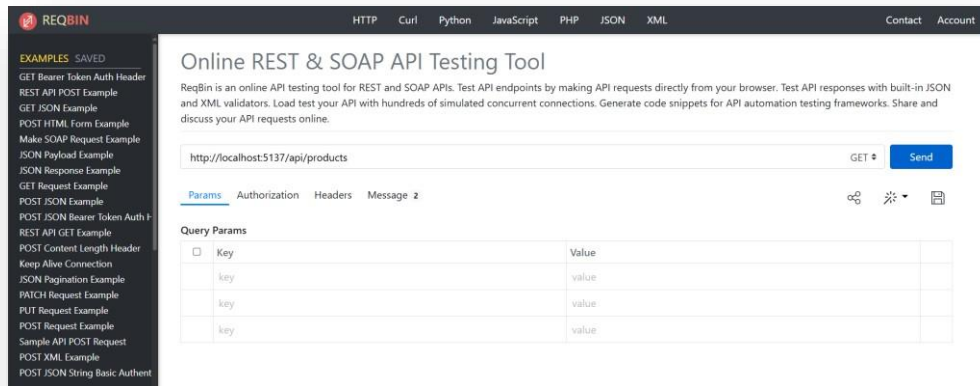
Use Swagger UI to test API endpoints directly from the documentation to verify they work as expected.

- Test the API by navigating to your browser
<http://localhost:<YourPortNumber>/api/products>.

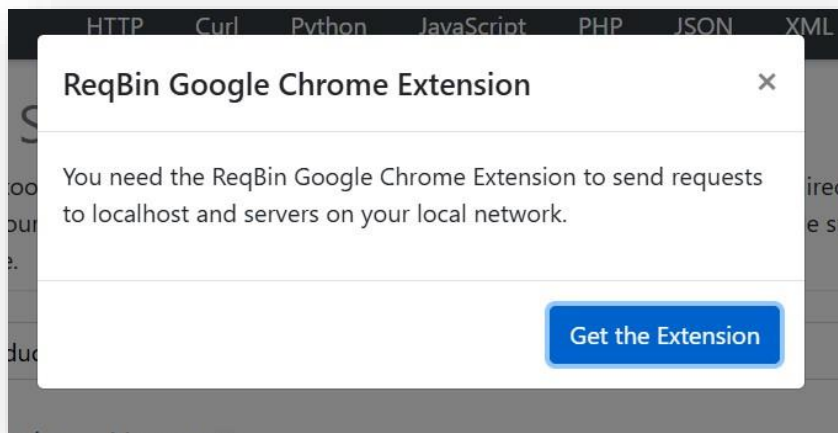


Part C: Testing the API from online Api tester (Optional)

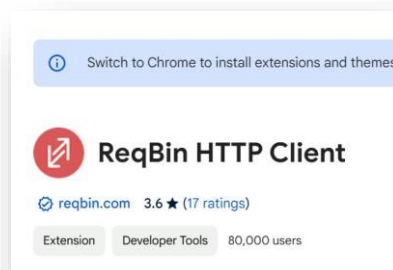
1. Open Postman (or any API testing tool). Here we have an online api tester called Reqbin.



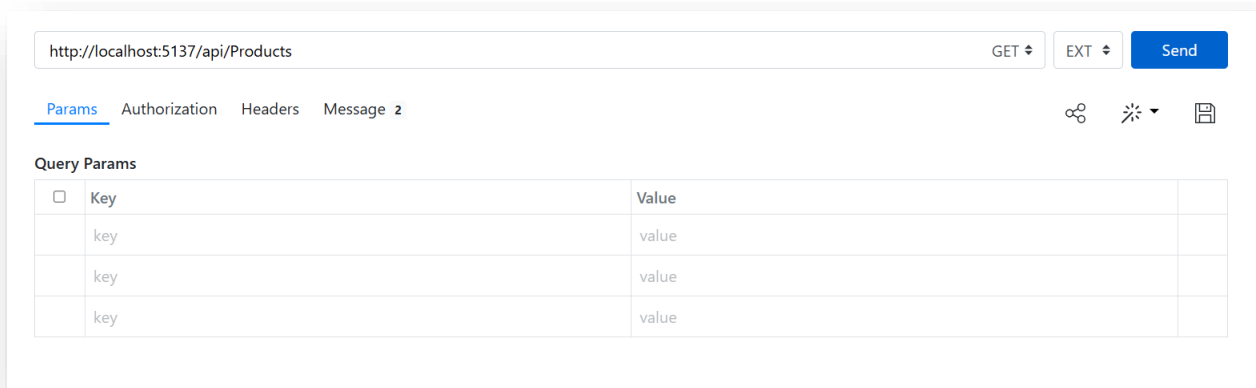
- To run the URL, it will ask you to download the ReqBin extension to connect with your localhost.



- This the extension reference.



2. After downloading, activate the extension and click on it to check the API online. Then, type your URL in the URL box and send a GET request.
<http://localhost:<YourPortNumber>/api/products>.



- Check the output, and verify that the response contains the list of products in JSON format.



Theory Questions:

1. Learn about all the HTTP methods and define each one, explaining their differences and use cases?
2. Explain the purpose of [ApiController] and [Route] attributes in WebAPI.
3. How does WebAPI handle JSON serialization by default?

Lab Tasks:

1. Modify the API to include a POST method for adding new products.
2. Implement validation to ensure no product is created without a name or price.

Lab # 11

Server-side technologies: LINQ queries, MVC Framework, C# Data First approach

Objective:

- To understand the basics of LINQ (Language Integrated Query) for querying data.
- To learn the Model-View-Controller (MVC) framework for web application development.
- To implement the Data First approach for database integration using C#.

Theory Background:

1. LINQ Queries:

Language-Integrated Query (LINQ) is a powerful set of technologies based on the integration of query capabilities directly into the C# language. Like classes, methods, and events, LINQ queries are the first-class language construct in C#.NET. The LINQ provides a consistent query experience to query objects (LINQ to Objects), relational databases (LINQ to SQL), and XML (LINQ to XML).

A query is an expression that retrieves data from a data source. Different data sources have different native query languages, for example SQL for relational databases and XQuery for XML. Developers must learn a new query language for each type of data source or data format that they must support.

LINQ simplifies this situation by offering a consistent C# language model for kinds of data sources and formats. In a LINQ query, you always work with C# objects. You use the same basic coding patterns to query and transform data in XML documents, SQL databases, .NET collections, and any other format when a LINQ provider is available.

All LINQ query operations consist of three distinct actions:

1. Obtain the data source.
2. Create the query.
3. Execute the query.

Below is the Example of above query operations expressed in source code

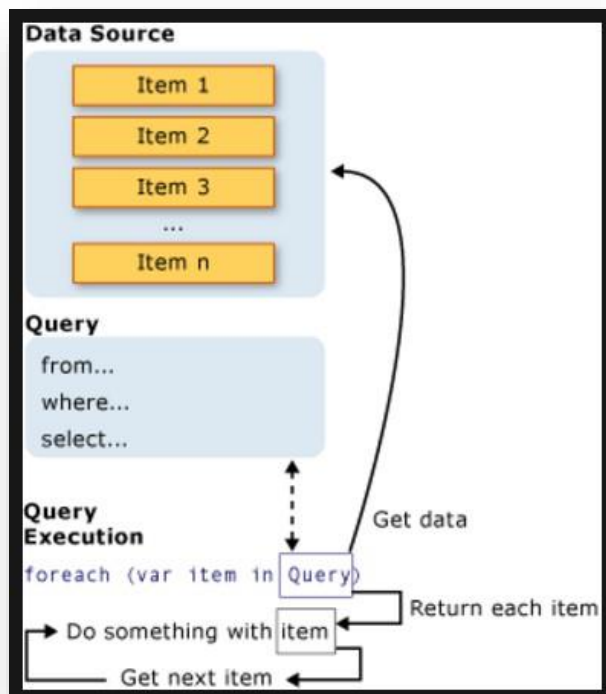
```
C#

// The Three Parts of a LINQ Query:
// 1. Data source.
int[] numbers = [ 0, 1, 2, 3, 4, 5, 6 ];

// 2. Query creation.
// numQuery is an IEnumerable<int>
var numQuery =
    from num in numbers
    where (num % 2) == 0
    select num;

// 3. Query execution.
foreach (int num in numQuery)
{
    Console.WriteLine("{0,1} ", num);
}
```

The entire query operation is depicted in the following figure. The way a query is executed in LINQ is different from the query itself. In other words, by creating a query variable, no data is retrieved.



Do u know?: LINQ to objects

"LINQ to Objects" refers to the use of LINQ queries with any IEnumerable or IEnumerable<T> collection directly. You can use LINQ to query any enumerable collections, such as List<T>, Array, or Dictionary<TKey,TValue>. The collection can be user-defined or a type returned by a .NET API. In the LINQ approach, you write declarative code that describes what you want to retrieve. LINQ to Objects provides a great introduction to programming with LINQ.

- LINQ queries offer three main advantages over traditional foreach loops:
 - They're more concise and readable, especially when filtering multiple conditions.
 - They provide powerful filtering, ordering, and grouping capabilities with a minimum of application code.
 - They can be ported to other data sources with little or no modification.

The more complex the operation you want to perform on the data, the more benefit you realize using LINQ instead of traditional iteration techniques.

Note:

IEnumerable Definition: Exposes an enumerator, which supports a simple iteration over a non-generic collection.

IEnumerable<T> Definition: Exposes the enumerator, which supports a simple iteration over a collection of a specified type.

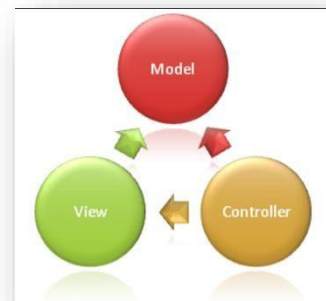
List<T> Definition: Represents a strongly typed list of objects that can be accessed by index. Provides methods to search, sort, and manipulate lists.

Array Definition: Provides methods for creating, manipulating, searching, and sorting arrays, thereby serving as the base class for all arrays in the common language runtime.

Dictionary<TKey,TValue> Definition: Represents a collection of keys and values.

2. MVC Framework:

The Model-View-Controller (MVC) framework is an architectural/design pattern that separates an application into three main logical components Model, View, and Controller. Every architectural element is designed to manage particular application development aspects. It separates the presentation layer from the business logic layer. Historically, desktop graphical user interfaces (GUIs) have made

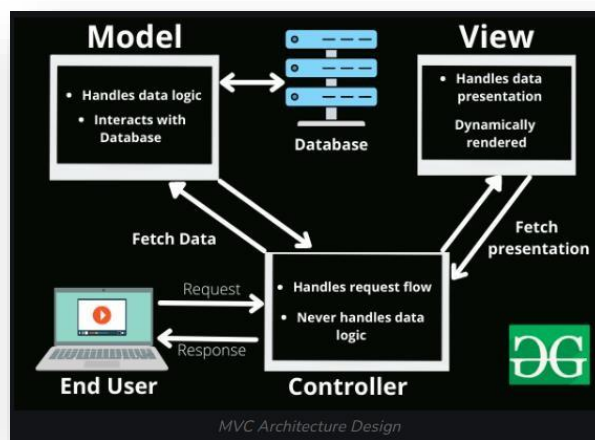


use of it. One of the most widely utilized industry-standard web development frameworks for producing scalable and adaptable projects these days is MVC. Mobile app design is another usage for it.

Note: Both the view and the controller depend on the model. However, the model depends on neither the view nor the controller. This is one of the key benefits of the separation. This separation allows the model to be built and tested independent of the visual presentation.

Features of MVC:

- It provides a clear separation of **business logic, UI logic, and input logic**.
- It offers full control over your HTML and URLs which makes it easy to design web application architecture.
- It is a powerful URL-mapping component using which we can build applications that have comprehensible and searchable URLs.
- It supports **Test Driven Development (TDD)**.



➤ **Components of MVC**

The MVC framework includes the following 3 components:

- Controller
- Model
- View

➤ **Controller:**

The controller serves as an intermediary, allowing the views and the model to be connected; it handles all business logic and incoming requests, manipulates data using the Model component, and communicates with the View to render the final output. The controller does not have to deal with data logic; it simply instructs the model on what to do.

Responsibilities:

- Receiving user input and interpreting it.
- Updating the Model based on user actions.
- Selecting and displaying the appropriate View.

Example: In a bookstore application, the Controller would handle actions such as searching for a book, adding a book to the cart, or checking out.

➤ **View:**

All of the application's UI functionality is implemented in the View component. For the user, it creates an interface. The data gathered by the model component—which is obtained via the controller rather than directly—is used to construct views. It communicates with the controller exclusively.

Responsibilities:

- Rendering data to the user in a specific format.
- Displaying the user interface elements.
- Updating the display when the Model changes.

Example: In a bookstore application, the View would display the list of books, book details, and provide input fields for searching or filtering books.

➤ **Model:**

All of the user's data-related logic is represented by the Model component. This could be any data pertaining to business logic or the data being moved between the View and Controller components. Data from the database can be added or retrieved by it. Since the controller is unable to communicate with the database on its own, it reacts to the controller's request. The model communicates with the database and provides the controller with the necessary information.

Responsibilities:

- Managing data: CRUD (Create, Read, Update, Delete) operations.
- Enforcing business rules.
- Notifying the View and Controller of state changes.

Example: In a bookstore application, the Model would handle data related to books, such as the book title, author, price, and stock level.

In short conclusion, The MVC is simply define as:

- **Model:** Manages the application's data and business logic.
- **View:** Displays data to the user and captures user input.
- **Controller:** Handles user input, updates the model, and returns the view.

Benefits of using MVC:

- **Organizes large-size web applications:** As there is segregation of the code among the three levels, it becomes extremely easy to divide and organize web application logic into large-scale applications (which are required to be managed by large teams of developers). The major advantage of using such code practices is that it helps to find specific portions of code quickly and allows the addition of new functionality with ease.
- **Supports Asynchronous Method Invocation (AMI):** Since the MVC architecture works well with JavaScript and its frameworks, it is no surprise that it also supports the use of Asynchronous Method Invocation (AMI), allowing developers to build faster loading web applications. It means that MVC applications can be made to work even with PDF files, site-specific browsers, and also for desktop widgets.
- **Easily Modifiable:** Using the MVC methodology allows easy modification of the entire application. Adding/updating the new type of views is simplified in the MVC pattern (as a single section is independent of the other sections). So, any changes in a certain section of the application will never affect the entire architecture. This, in turn, will help to increase the flexibility and scalability of the application.
- **Faster Development Process:** As there is segregation of the code among the three levels, developing web applications using the MVC model allows one developer to work on a particular section (say, the view) while another can work on any other section (say, the controller) simultaneously. This allows for easy implementation of business logic as well as helps to accelerate the development process fourfold. It has been observed that when compared to other development models, the MVC model ends up showing higher development speeds (up to three times).
- **Returns data without formatting:** By returning un-formatted data, the MVC framework empowers you to create your own view engine. For example, any type of data can be formatted using HTML, but with the MVC framework, you can also format the data using the Macromedia Flash or Dream viewer. It is helpful for the developers because the same components can be re-used with any interface.
- **Supports TTD (test-driven development):** A major advantage of the MVC pattern is that it simplifies the testing process by a great deal. It makes it easier to debug large-scale applications as multiple levels are structurally defined and properly written in the application. Thus making it trouble-free to develop an application with unit tests.
- **Multiple Views:** In the MVC architecture, developing different view components for your model component is easily achievable. It empowers you to develop different view components, thus limiting code duplication as it separates data and business logic.

- **SEO-Friendly Platform:** The MVC platform hugely supports the development of SEO-friendly web applications. To generate more visits from a particular application, MVC provides an easy way out to develop SEO-friendly RESTful URLs.

3. Data First Approach:

An existing database structure serves as the foundation for the Data First approach, a methodology used in application development. With this method, the database has already been created and filled with data, tables, and relationships. The main goal is to create comparable models in C# in order to link this existing database to your application. These models serve as a link between the database and the application logic, representing the database schema.

Developers can use technologies like Entity Framework (EF) to scaffold models straight from the database by employing the Data First methodology. Classes created by the scaffolding process replicate the database's tables, columns, and relationships. This guarantees consistency between the database structure and the application code in addition to saving time.

This approach is particularly useful in scenarios where:

- The database is designed and managed by a separate team or department.
- You are integrating a legacy database into a modern application.
- There is a need to interact with a well-established and stable database schema.

With the Data First approach, developers can immediately start querying and manipulating data using technologies like LINQ (Language Integrated Query) while maintaining a clean and structured codebase. It streamlines the process of incorporating databases into applications and allows for seamless interaction with complex data systems.

Additional Resource: *Want to learn more about the Data First approach in depth and with practical examples? Visit this*

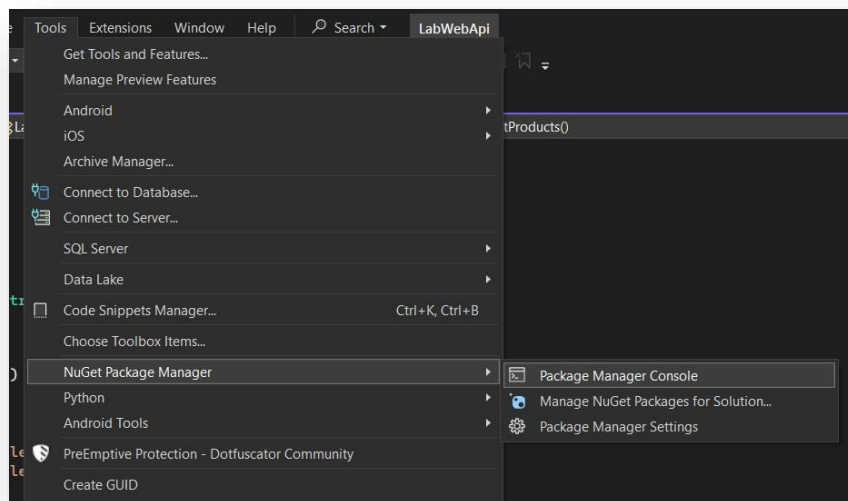
website: (<https://dotnettutorials.net/lesson/entity-framework-core-database-first-approach/>)

Experiment Steps:

In this lab, we will continue working on the .NET project created in **Lab 02** to implement the **Data First Approach** for database integration.

Step 1: Install Required NuGet Packages

1. Open the **.NET project** created in Lab 02.
2. Install the following NuGet packages to enable Entity Framework Core and SQL Server support:
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools
3. To install, open the **NuGet Package Manager Console** from **Tools > NuGet Package Manager > Package Manager Console** and run these commands:



- **Install-Package Microsoft.EntityFrameworkCore.SqlServer**
- **Install-Package Microsoft.EntityFrameworkCore.Tools**

Step 2: Connect to the Database

1. Create a database (e.g., LabDatabase) in SQL Server Management Studio.
 - Add a table named Students with the following schema:
 - **StudentId** (int, Primary Key)
 - **Name** (nvarchar(100))
 - **Age** (int)
- Here is a simple SQL query to create the basic database.

```
-- Create the database
CREATE DATABASE LabDatabase;
GO

-- Use the newly created database
USE LabDatabase;
GO

-- Create the Students table
CREATE TABLE Students (
    StudentId INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(100) NOT NULL,
    Age INT NOT NULL
);
GO
```

Also, don't forget to insert some dummy data.

```
INSERT INTO Students (Name, Age)
VALUES
('John Doe', 20),
('Jane Smith', 22),
('Alice Johnson', 19),
('Bob Brown', 21),
('Charlie White', 23);
```

2. Open the **appsettings.json** file in your .NET project and add the database connection string under the AllowedHosts entry section:

```
"ConnectionStrings": {
  "DefaultConnection":
  "Server=YOUR_SERVER_NAME;Database=LabDatabase;Trusted_Connection=True;"
}
```

Note: Don't Forget to add a comma after AllowedHosts and also don't forget to your server name in server option like this "(LocalDb)\\mydatabase".

Step 3: Scaffold Models from the Database

1. Use the **Entity Framework Core Tools** to generate models from the database.
2. In the **Package Manager Console**, run the following command to scaffold the Students table:

Scaffold-DbContext

```
"Server=(localdb)\mydatabase;Database=LabDatabase;Trusted_Connection=True;  
" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models
```

3. This will create a folder named **Models** containing the C# classes representing your database schema.

Step 4: Update the Controller

1. Create the Controller name as StudentsController in Controller folder.
2. Update the code in Student Controller.

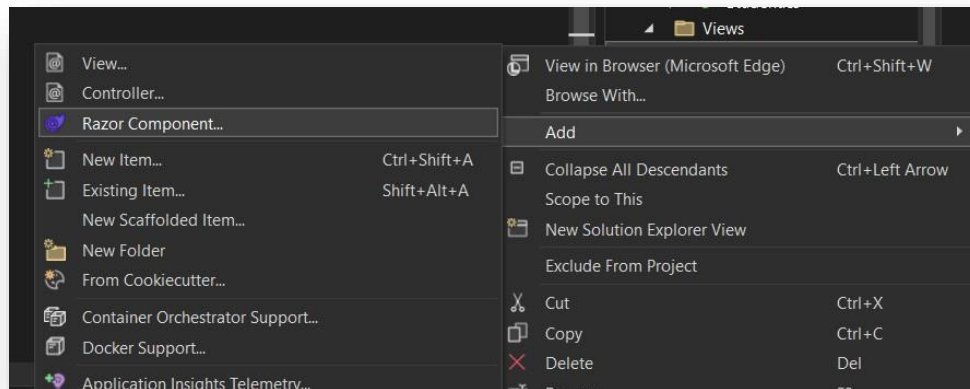
```
using LabWebApi.Models; // Namespace for the DbContext and Student model  
using Microsoft.AspNetCore.Mvc; // Namespace for Controller base class and  
IActionResult  
using System.Linq; // Namespace for LINQ methods
```

```
namespace LabWebApi.Controllers  
{  
    public class StudentsController : Controller  
    {  
        private readonly LabDatabaseContext _context;  
  
        public StudentsController(LabDatabaseContext context)  
        {  
            _context = context;  
        }  
  
        public IActionResult Index()  
        {  
            var students = _context.Students.ToList();  
            return View(students);  
        }  
    }  
}
```

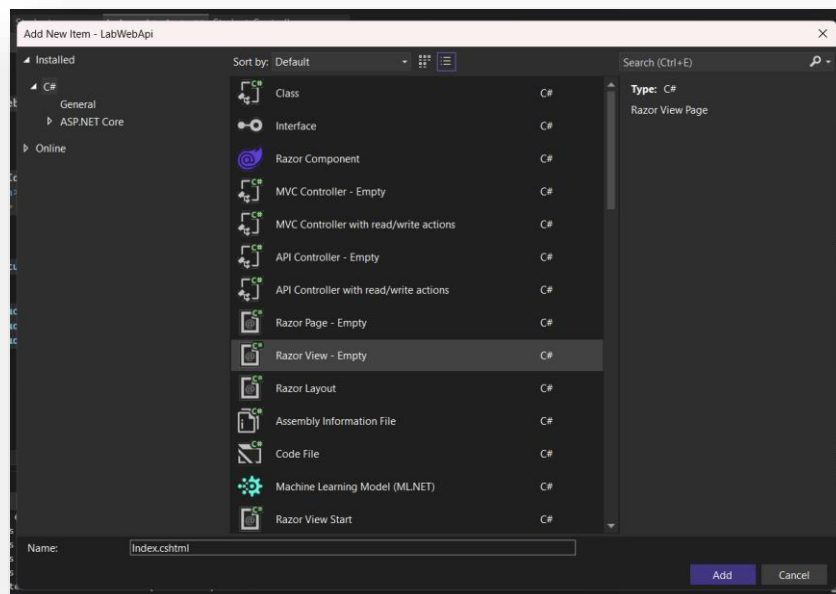
Step 5: Create the Views Folder and Razor View

1. Click on the Solution Explorer, create a folder named "Views", then inside the Views folder, create another folder named "Students".

- Then, Double click on the Students folder and add Razor Components.



- Finally, create a new Razor view named Index.cshtml inside the Views/Students folder.



- Add the following code to display student data in a table:

```
@model IEnumerable<LabWebApi.Models.Student>
<h2>Student List</h2>
<table class="table">
  <thead>
    <tr>
      <th>StudentId</th>
      <th>Name</th>
```



```

        <th>Age</th>
    </tr>
</thead>
<tbody>
    @foreach (var student in Model)
    {
        <tr>
            <td>@student.StudentId</td>
            <td>@student.Name</td>
            <td>@student.Age</td>
        </tr>
    }
</tbody>
</table>

```

Step 6: Update the Program.cs

1. Open the Program.cs file and then update the code.

```

using Microsoft.EntityFrameworkCore;
var builder = WebApplication.CreateBuilder(args);
// Add services to the container.
builder.Services.AddControllersWithViews(); // Enable MVC and Afl
controllers

builder.Services.AddDbContext<LabWebApi.Models.LabDatabaseContext>(opti
ons =>

options.UseSqlServer(builder.Configuration.GetConnectionString("Default
Connection")));

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseStaticFiles(); // For serving static files like CSS and
JavaScript
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

Explanation of Changes in Program.cs:

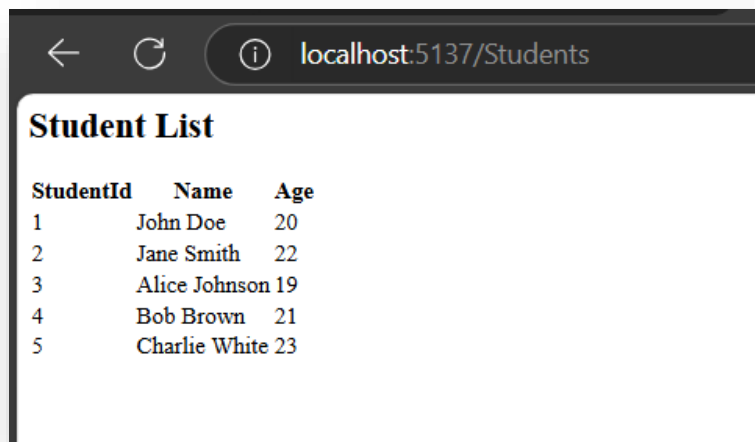
The changes in Program.cs are made to configure the application for both API and MVC functionality, enabling it to handle API requests and serve Razor views. By adding `AddControllersWithViews()`, the application supports both MVC controllers and API endpoints, allowing for dynamic web pages and JSON-based APIs.

The `AddDbContext` method registers the `LabDatabaseContext` with the Dependency Injection container and configures it to use SQL Server as the database provider, ensuring seamless interaction with the database. Middleware components like `UseStaticFiles()` enable serving static assets such as CSS and JavaScript, while `MapControllerRoute` defines the default routing pattern for controllers and actions.

Additionally, Swagger is configured for API documentation, providing an interactive interface for testing endpoints during development. These configurations ensure the application is robust, secure, and capable of serving both data and user interfaces effectively.

Step 7: Test the Application

1. Run the application in Visual Studio (**Ctrl + F5**).
2. Navigate to `/Students` in your browser (e.g., `http://localhost:<YourPort>/Students`).
3. Verify that the list of students from the database is displayed on the web page.

**Theory Questions:**

1. What is the purpose of LINQ in C#?
2. Explain the components of the MVC framework.
3. How does the Data First approach differ from the Code First approach?

Tasks for Students:

1. Add a feature to insert new student records using a form in the MVC application.
2. Use LINQ queries to filter students based on age (e.g., display students older than 18).

Lab # 12

Server-side technologies: C# Data First Approach with Web APIs – Web API Consumption

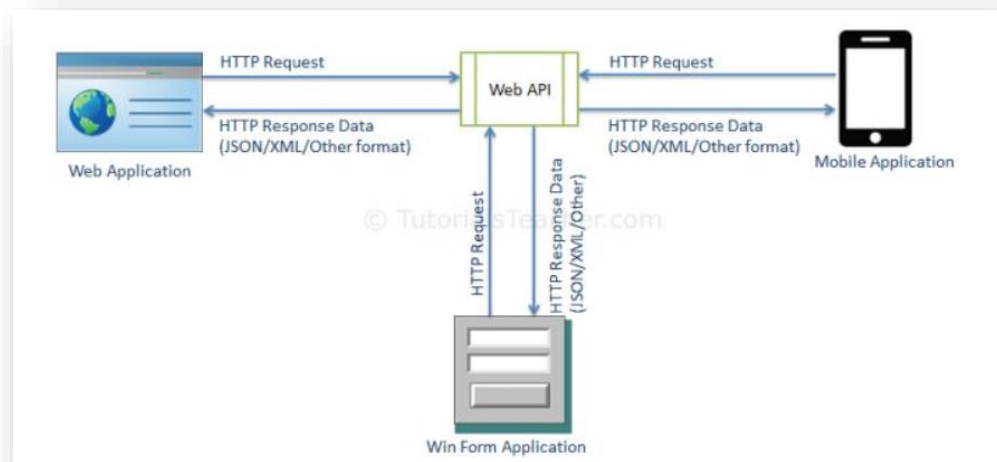
Objective:

- To understand and implement the Data First Approach in C# for database integration and to create and consume Web APIs.

Theory Background:

1. Web APIs:

Web API is a programming interface/application type that provides communication or interaction between software applications. Web API is often used to provide an interface for websites and client applications to have data access. Web APIs can be used to access data from a database and save data back to the database.



Web APIs (Application Programming Interfaces) allow applications to communicate with each other over the HTTP protocol. APIs expose endpoints that can be consumed by clients to perform CRUD (Create, Read, Update, Delete) operations on the server. ASP.NET Web API is a framework that make it easy to build HTTP web service that reaches a broad range of clients, including browser, mobile applications, Desktop application and IOTs.

➤ HTTP Methods

The following HTTP methods are most commonly used in a REST based architecture.

1. GET – Provides a read only access to a resource.
2. PUT – Used to create a new resource.
3. DELETE – Used to remove a resource.
4. POST – Used to update an existing resource or create a new resource.

API	Description	Request body	Response body
GET /api/todoitems	Get all to-do items	None	Array of to-do items
GET /api/todoitems/{id}	Get an item by ID	None	To-do item
POST /api/todoitems	Add a new item	To-do item	To-do item
PUT /api/todoitems/{id}	Update an existing item	To-do item	None
DELETE /api/todoitems/{id}	Delete an item	None	None

➤ Key Features of ASP.NET for Building Web APIs

- **Simple serialization:** ASP.NET was designed for modern web experiences. Endpoints automatically serialize your classes to properly formatted JSON out of the box. No special configuration is required. Of course, serialization can be customized for endpoints that have unique requirements.
- **Authentication and authorization:** Secure API endpoints with built-in support for industry standard JSON Web Tokens (JWT). Policy-based authorization gives you the flexibility to define powerful access control rules all in code.
- **Routing alongside your code:** ASP.NET lets you define routes and verbs inline with your code, using attributes. Data from the request path, query string, and request body are automatically bound to method parameters.
- **Designed with security in mind:** You don't deploy your apps without security, so why test them without security? ASP.NET provides first class support for HTTPS out of the box. Automatically generate a test certificate and easily import it to enable local HTTPS so you run, and debug, your apps the way they are intended to be... secured.

- **Fast and scalable:** APIs built with ASP.NET Core perform faster than any popular web framework in the independent TechEmpower benchmarks.

The flexible nature of the ASP.NET Web API framework is one of its main features. It doesn't stick to one architectural style, however it is often known and utilized for creating RESTful services, or services that follow the guidelines of Representational State Transfer (REST). The framework offers the resources and adaptability needed to develop services that follow to other architectures, such SOAP (Simple Object Access Protocol) and RPC (Remote Procedure Call). This makes ASP.NET Web API a strong and flexible tool for creating a variety of service-oriented applications that are suited to particular project requirements because developers are not limited to a particular design paradigm when utilizing it.

*I think you got the point, Right? but we should learn about **RESTful** services also since they are a key tech trend. So, let's dive into it too!*

REST stands for **Representational State Transfer** and was first introduced in 2001. It is an architectural pattern for building APIs that use **HTTP** as a communication protocol. REST defines a set of constraints that guide the design of scalable, reliable, and efficient systems.

Key REST Constraints

1. Client-Server

The REST architecture enforces a clear separation between the client and the server.

- The **client** sends a request, and the **server** processes the request and sends a response.
- This separation ensures that the client-side logic (e.g., UI updates) and server-side logic (e.g., data processing) evolve independently, enhancing scalability and maintainability.

2. Stateless Communication

Each request from the client to the server must be **stateless**, meaning no client-specific data is stored on the server between requests.

- Every request should include all the information the server needs to process it.
- This ensures that each request is independent, reducing server memory usage and enabling better scalability.

3. Cacheable

REST supports **caching** to optimize performance and reduce unnecessary server load.

- Data that doesn't change frequently, like a list of products or departments, can be cached.
- The server specifies how long the data is valid, ensuring the client can reuse the response without making repeated requests.
- This reduces network traffic and improves system efficiency.

4. Uniform Interface

The interaction between the client and server is defined by a **uniform interface**, making it easy to understand and use.

- In REST, **resources** are the central concept. Resources typically represent entities like Product, Employee, etc.
- Each resource is identified by a **URI (Uniform Resource Identifier)**.

Example:

URI	HTTP Verb	Action	Description
/Employees	GET	Retrieve a list of employees	Fetches all employee records.
/Employees/1	GET	Retrieve a single employee	Fetches the employee with ID 1.
/Employees	POST	Create a new employee	Adds a new employee to the system.
/Employees/1	PUT	Update an employee	Updates details for employee ID 1.
/Employees/1	DELETE	Delete an employee	Removes the employee with ID 1.

- **HTTP Verbs** like **GET**, **POST**, **PUT**, and **DELETE** define the action to perform on the resource.

Why Use RESTful Services?

- **Scalability**: Separation of client and server allows for independent scaling.
- **Performance**: Stateless communication and caching ensure fast and efficient interactions.
- **Simplicity**: The uniform interface makes APIs intuitive and easy to use.
- **Interoperability**: REST APIs can be consumed by clients across platforms and technologies, making them versatile.

2. Web API Consumption:

Web API consumption refers to the process where client-side applications or tools interact with Web APIs to leverage their functionalities. APIs (Application Programming Interfaces) expose endpoints that allow clients to send HTTP requests and receive responses, enabling seamless communication between systems.

When consuming an API, the client application sends requests to specific endpoints using HTTP verbs such as **GET**, **POST**, **PUT**, or **DELETE**, depending on the intended action. The server processes these requests and returns structured responses, often in **JSON** or **XML** format, which the client application can then use to display data, perform operations, or trigger additional workflows.

For example:

- A **mobile app** might consume an API to fetch weather data for a location.
- A **web application** might interact with a payment gateway API to process transactions.
- A **desktop software** might use a cloud storage API to upload and retrieve files.

API consumption enables modular and distributed system design by allowing the client to focus on user interactions and interface logic while delegating tasks like data processing, storage, and business logic to the API. Tools like **Postman** or programming languages with libraries like **HttpClient** in C# make it easier to test and consume APIs effectively.

In the upcoming experiment, we will learn how to consume Web APIs by making HTTP requests, processing the responses, and integrating the data into a client-side application. This hands-on approach will demonstrate how Web API consumption works in practice, providing a solid foundation for building scalable and efficient applications.

3. C# Data First Approach with Web Api:

The **C# Data First Approach** focuses on generating models directly from an existing database. This method is particularly useful when the database is pre-designed, and the application needs to integrate with it for data manipulation and retrieval. By scaffolding the database schema into C# models, developers can seamlessly interact with tables and relationships using Entity Framework Core.

Note that in Lecture 02, we covered and used the Data First Approach, performing basic CRUD operations and scaffolding database models into our project. By incorporating Web APIs, this presentation will build upon that base and extend the idea. Through the creation of APIs, database activities will be provided as endpoints, allowing external applications to use HTTP requests to access and interact with the data.

This combination of Data First Approach and Web APIs enhances modularity, scalability, and reusability, allowing the system to serve as both a backend service provider and a client-facing API.

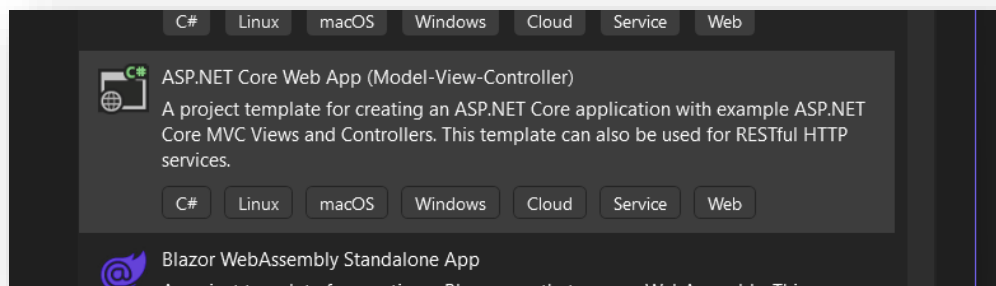
Want to master Web APIs? Here are some additional resources for you:

⇒ <https://dotnettutorials.net/course/asp-net-web-api/>

Experiment Steps:

Step 1: Setup the .NET Project

1. Create a new ASP.NET Core project in Visual Studio using the **ASP.NET Core Web Application (Model-View-Controller)** template.



2. Name the project “**LabWebAPIConsumption**”.
3. Install the required NuGet packages:
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools
 - System.Net.Http.Json

Here, is the commands:

- **Install-Package Microsoft.EntityFrameworkCore.SqlServer**
- **Install-Package Microsoft.EntityFrameworkCore.Tools**
- **Install-Package Swashbuckle.AspNetCore**

Step 2: Configure the Database and Models

1. Create a table in that same database (e.g., LabDatabase) as we create before:

SQL Query:

```
CREATE TABLE Products (  
    ProductId INT PRIMARY KEY IDENTITY(1,1),  
    Name NVARCHAR(100) NOT NULL,  
    Price DECIMAL(18,2) NOT NULL  
);  
  
Select * from Products
```

Also, Don't forget to add some sample data

```
INSERT INTO Products (Name, Price) VALUES  
( 'Laptop' , 899.99),  
( 'Smartphone' , 499.99),  
( 'Headphones' , 79.99),  
( 'Smart Watch' , 199.99),  
( 'Keyboard' , 49.99),  
( 'Mouse' , 29.99),  
( 'Monitor' , 249.99),  
( 'Tablet' , 349.99),  
( 'External Hard Drive' , 129.99),  
( 'Webcam' , 69.99);
```

2. Open the appsettings.json file in your .NET project and add the database connection string under the AllowedHosts entry section.

```
"ConnectionStrings": {  
    "DefaultConnection":  
    "Server=(LocalDb)\mydatabase;Database=LabDatabase;Trusted_Connection=  
    True;"  
}
```

3. Scaffold the database using the Data First Approach to make your database model:

Scaffold-DbContext

```
"Server=(LocalDb)\mydatabase;Database=LabDatabase;Trusted_Connection=True;" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models
```

Step 3: Create the Web API

- Add a new controller named ProductsController to handle API requests:

```
using LabWebAPIConsumption.Models;  
using Microsoft.AspNetCore.Mvc;
```

```
namespace LabWebAPIConsumption.Controllers  
{
```

```
    [ApiController]
```

```
    [Route("api/[controller]")]
```

```
    public class ProductsController : ControllerBase  
    {
```

```
        private readonly LabDatabaseContext _context;
```

```
        public ProductsController(LabDatabaseContext context)  
        {  
            _context = context;  
        }
```

```
        [HttpGet]
```

```
        public IActionResult GetProducts()  
        {  
            var products = _context.Products.ToList();  
            return Ok(products);  
        }
```

```
        [HttpPost]
```

```
        public IActionResult AddProduct([FromBody] Product product)  
        {  
            _context.Products.Add(product);  
            _context.SaveChanges();  
            return CreatedAtAction(nameof(GetProducts), new { id = product.ProductId }, product);  
        }  
    }  
}
```

- And Update your Program.cs:

```
using Microsoft.EntityFrameworkCore;
var builder = WebApplication.CreateBuilder(args);
// Add services to the container.
builder.Services.AddControllersWithViews(); // Enable MVC and API controllers
builder.Services.AddDbContext<LabWebAPIConsumption.Models.LabDatabaseContext>(options => options.UseSqlServer(builder.Configuration.GetConnectionString("Default Connection")));

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection();
app.UseStaticFiles(); // For serving static files like CSS and JavaScript
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

- Then Test the Web API using **Swagger** (It's already configured in Program.cs)
- Navigating to the URL: (<https://localhost:<YourPort>/swagger/index.html>)



```
"price": 1000
},
{
  "productId": 13,
  "name": "New Full Sleeve shirt",
  "price": 1200
}
]
```

Step 4: Consume the Web API

Add a new MVC controller named **ProductsConsumerController** to interact with the Web API:

```
using LabWebAPIConsumption.Models;
using Microsoft.AspNetCore.Mvc;
using System.Net.Http.Json;
using System.Threading.Tasks;
```

```
public class ProductsConsumerController : Controller
{
    private readonly LabDatabaseContext _context;

    public ProductsConsumerController(LabDatabaseContext context)
    {
        _context = context;
    }

    public IActionResult Index()
    {
        var products = _context.Products.ToList();
        return View(products);
    }
}
```

Create a Razor view (Index.cshtml) under Views/ProductsConsumer to display products:

```
@{
    @model IEnumerable<LabWebAPIConsumption.Models.Product>

    <h2>Products</h2>
    <table class="table">
        <thead>
            <tr>
                <th>Product ID</th>
                <th>Name</th>
                <th>Price</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var product in Model)
            {
                <tr>
                    <td>@product.ProductId</td>
                    <td>@product.Name</td>
                    <td>@product.Price</td>
                </tr>
            }
        </tbody>
    </table>
}
```

Products		
Product ID	Name	Price
1	Laptop	899.99
2	Smartphone	499.99
3	Headphones	79.99
4	Smart Watch	199.99
5	Keyboard	49.99
6	Mouse	29.99
7	Monitor	249.99
8	Tablet	349.99
9	External Hard Drive	129.99
10	Webcam	69.99
11	string	0.00
12	Anas Ahmed	1000.00
13	New Full Sleeve shirt	1200.00
14	Long Shirts	1800.00

Step 5: Test the Application

1. Run the application and navigate to /ProductsConsumer to view the list of products fetched from the Web API.
2. Use Swagger to test the /api/products endpoints for adding new products.

Review Questions

1. Explain how Web API endpoints are created and consumed.
2. What is the advantage of using HTTPClient for API consumption?

Tasks for Students

1. Add functionality to update and delete products via the Web API.
2. Create a form in the consumer application to allow adding new products to the database through the API.

Lab # 13

Deployment, Operation & Maintenance of Web Applications

Objective:

- Gain a comprehensive understanding of deploying web applications and the related infrastructure.
- Learn the theoretical foundations of web application operation post-deployment.
- Develop insights into the best practices and challenges associated with web application maintenance.
- Recognize the importance of systematic monitoring, scaling, and debugging in production environments.

Theory Background:

Introduction to Deployment

What is Web Application Deployment?

Deployment refers to the process of transferring a web application from a local development environment to a live server where users can access it over the internet. It ensures that the application becomes available for public or private use. Web deployment involves setting up necessary infrastructure, ensuring compatibility with the hosting platform, and managing application settings such as database connections and environment-specific configurations.

Types of Deployment Environments:

1. **Local Hosting:** Applications are hosted on local machines using tools like IIS (Internet Information Services) or XAMPP. This is primarily used for testing purposes.
2. **On-premises Servers:** Applications are hosted on physical servers owned and maintained by the organization. This requires a dedicated team to manage the servers and their security.
3. **Cloud Hosting:** Modern applications often use cloud platforms like Microsoft Azure, AWS, or Google Cloud due to their scalability, reliability, and cost-effectiveness. Cloud hosting allows dynamic resource allocation based on the application's needs.
4. **Hybrid Deployment:** A combination of on-premises and cloud hosting where critical data is stored on-premises while other parts of the application operate in the cloud.

Deployment Process

Step 1: Preparation

Before deployment, several steps must be completed:

- **Testing:** Ensure that the application is tested thoroughly in a local development environment.
- **Configuration:** Adjust settings specific to the hosting environment, such as database connection strings, API keys, and authentication protocols.
- **Environment Variables:** Replace hard-coded values with environment-specific variables to improve security and flexibility.
- **Build Optimization:** Create a release build of the application that is optimized for production, typically by using tools like Visual Studio or Node.js build tools.

Step 2: Publishing the Application

Publishing involves preparing the application files for deployment:

- **Visual Studio Example:** In Visual Studio, right-click on the project and select "Publish." You can publish to a folder or directly to a hosting environment like Azure.
- **Content Delivery:** Static assets like CSS, JavaScript, and images are bundled and minified to ensure faster load times in production.

Step 3: Deploying to a Server

The next step is moving the published files to a server:

- **Local Server Deployment:** Use IIS to host the application on a local or organizational server. The steps include creating a new site in IIS, configuring bindings, and pointing the site to the application folder.
- **Cloud Deployment:** If deploying to Azure, create an App Service instance, configure its settings, and upload the application files through Visual Studio or Azure DevOps pipelines.
- **Domain Configuration:** Link a domain name to the application for public access. This involves setting up DNS records to point to the server or cloud platform.

Understanding Application Operation

How a Web Application Works Post-Deployment

1. **User Request:** The user accesses the application by entering the URL in a browser.
2. **Routing:** The request is routed to the server, which determines the appropriate controller and action method to execute.
3. **Server-Side Processing:** The server processes the request, interacts with the database if required, and prepares a response.
4. **Response:** The server sends the processed data (often in the form of HTML,

JSON, or XML) back to the browser.

5. **Browser Rendering:** The browser renders the response for the user to view and interact with.

Importance of Monitoring and Logging

Monitoring and logging are critical for ensuring smooth operation. They allow developers to track application behavior, detect errors, and gain insights into user activity. Popular logging tools include Serilog, ELK Stack, and Application Insights. These tools help track metrics like:

- HTTP response times.
- Error rates and stack traces.
- Database query performance.

Maintenance of Deployed Applications

Key Maintenance Tasks

1. **Monitoring Application Health:**

- Use tools like Azure Monitor or AWS CloudWatch to track the uptime and performance of your application.
- Configure alerts for critical issues such as server downtime or high response times.

2. **Bug Fixes and Updates:**

- Fix bugs reported by users or detected in logs.
- Apply security updates to protect against vulnerabilities.
- Implement a version control system to track changes and roll back if necessary.

3. **Scaling:**

- Scale vertically by adding resources (CPU, memory) to the server.
- Scale horizontally by adding more servers to handle increased traffic.

4. **Database Maintenance:**

- Regularly back up the database to prevent data loss.
- Optimize queries and indexes to ensure fast response times.
- Monitor database storage to avoid running out of space.

Common Challenges and Solutions

1. **Database Connection Failures:**

- Ensure proper configuration of connection strings.
- Use connection pooling to manage multiple requests efficiently.

2. **Server Downtime:**

- Set up load balancers to distribute traffic and avoid server overload.
- Use auto-scaling features of cloud platforms to adjust resources dynamically.

3. Security Vulnerabilities:

- Use HTTPS to encrypt data.
- Regularly update dependencies to patch known vulnerabilities.
- Implement role-based access control to restrict unauthorized access.

Case Study

Scenario:

A company has deployed an e-commerce web application to Azure. The deployment involved configuring the application to connect to an Azure SQL database, setting up the application on Azure App Services, and enabling basic monitoring tools for tracking errors and performance. However, shortly after deployment, the following issues were reported:

1. **Database Connection Errors:** The application frequently experienced intermittent downtime due to failed database connections. The database connection string was configured manually, but no redundancy measures were put in place.
2. **Performance Degradation During Peak Hours:** The application performed well during normal usage but slowed significantly when traffic spiked, especially during promotional events. This led to user complaints and abandoned carts. No caching mechanisms or scaling configurations were implemented at deployment.
3. **Incomplete Logs for Troubleshooting:** When errors occurred, the application logs provided insufficient details, making it challenging for the technical team to identify and resolve the root causes. Only basic logging was enabled, without structured or centralized log storage.

Task for Students:

1. **Analyze the Database Connection Errors**
 - Identify potential causes of the intermittent database connectivity issues.
 - Propose solutions, such as implementing retry policies, database redundancy, and better connection string configuration.
2. **Address the Performance Degradation During Peak Hours**
 - Explain why the application may struggle during high traffic.
 - Suggest strategies to enhance performance, including implementing caching, enabling auto-scaling in Azure, and optimizing database queries.
3. **Resolve the Incomplete Logs Issue**
 - Describe the importance of detailed and structured logs for monitoring and troubleshooting.
 - Suggest a plan to implement robust logging, including using structured logging frameworks (e.g., Serilog) and integrating logs with Azure Monitor or Application Insights.

Deliverable:

Submit a detailed explanation of:

- The root causes of the issues.
- Step-by-step solutions to address each problem.
- A strategy to prevent similar issues in the future, such as proper configuration, testing, and proactive monitoring.